

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING (SCOPE)

REVIEW -0

ASSESMENT -8

BACSE202 LAB COMPONENT PROJECT

B. Tech [CSE & CSE-With Specialisation]

AY:2026-27

APPENDIX 1
(A typical Specimen of a cover page)

TITLE OF PROJECT REPORT:

MODERN AIRLINE LOGISTICS & SUSTAINABILITY MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

25BCE2957 – Pranali Paygonda Patil

B.Tech Computer Science and Engineering
Vellore Institute of Technology
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING (SCOPE)

AY: 2026-27

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

CONTENT

S.NO	TITLE	PAGE NO
1	ABSTRACT OF THE PROJECT	6
2.	ACKNOWLEDGEMENT	7
3.	INTRODUCTION OF THE PROJECT	8
4.	BACKGROUND OF THE PROJECT WORK	9
5.	HARDWARE AND SOFTWARE REQUIREMENTS	10
6.	E-R DIAGRAM	12
7.	DESCRIPTION OF THE DATABASE DESIGN	13
8.	Relationships / Cardinality Table	15
8.	RELATIONAL SCHEMA	16
9.	EXECUTION OF THE SQL QUERY WITH OUTPUT SCREENSHOT TABLE CREATION WITH ALL CONSTRAINTS INSERTION UP TO 5 ROWS DELETION UPDATION (ON EACH TABLE) SELECT SINGLE ROW FUNCTIONS SQL FUNCTIONS GROUP BY HAVING CLAUSE SUB QUERY JOIN QUERY	19
8		
9		

CONTENT (CONTINUED)

S.NO	TITLE	PAGE NO
	LIST OF FIGURES	5
10.	FRONT END DESIGN	41
11.	BACK END TABLES & CONNECTIVITY	42
12.	PRACTICAL / BUSINESS / SUSTAINABILITY	45
13.	CODE AND APPLICATION SCREENSHOTS	47–79
14.	CONCLUSION / FUTURE SCOPE	80

LIST OF FIGURES

Figure 1. ER Diagram

Figure 2. Overall System Design and Architecture

Figure 3. Workflow of the Project

Figure 4. Sign-in page

Figure 5. Overview dashboard

Figure 6. Account registration page

Figure 7. Flight schedule

Figure 8. Oracle flight rows before insertion

Figure 9. Create flight form

Figure 10. Oracle flight rows after insertion

Figure 11. Flight details

Figure 12. Update flight form

Figure 13. Oracle updated flight record

Figure 14. Delete confirmation prompt

Figure 15. Fleet module

Figure 16. Crew roster module

Figure 17. Cargo tracking module

Figure 18. Passengers and loyalty module

Figure 19. Disruptions module

Figure 20. Emissions module

Figure 21. Updated overview dashboard

ABSTRACT

The **Modern Airline Logistics and Sustainability Management System** is a database-driven system designed to manage and integrate major airline logistics operations through a centralized relational database. Airline operations involve the continuous coordination of aircraft, flights, crew, passengers, specialized cargo, emissions and unexpected operational disruptions. Managing these activities through disconnected systems can lead to data inconsistency, delays, cargo-handling errors and difficulty in obtaining timely operational information.

The proposed system uses a relational database to maintain accurate and structured information regarding the airline fleet, flight schedules, crew assignments, passenger loyalty tiers, baggage limits, specialized cargo and flight-level sustainability records. It also incorporates cold-chain cargo monitoring for temperature-sensitive shipments, carbon-emission logging for sustainability analysis, and disruption management for recording delays caused by weather, technical issues, air-traffic control, crew-related problems and security events.

The database consists of nine interconnected entities with appropriate primary keys, foreign keys, NOT NULL, UNIQUE, CHECK and DEFAULT constraints. The schema is designed according to relational database principles and aims to reduce redundancy while preserving referential integrity.

SQL operations including DDL, DML, SELECT queries, single-row functions, aggregate functions, GROUP BY, HAVING, subqueries and JOIN operations are implemented to demonstrate how the database can support both routine airline operations and management-level decision making.

ACKNOWLEDGEMENT

I would like to thank Dr. BOOMINATHAN P (HOD) for reviewing the entire project and guiding us as the team leader, and also for helping me coordinate my project. After the completion of my project, I would like to take this chance to express my sincere gratitude to my project guide, Dr. KAVITHA G, who has guided me thoroughly throughout my project development. I would like to express my gratitude to all our friends and teachers, who helped and cooperated directly or indirectly in the accomplishment of this project. Heartfelt appreciation to those who helped me through cooperative acts and logical devices to make this project effective and user-friendly.

INTRODUCTION OF THE PROJECT

Airline logistics is a complex and highly coordinated operational environment in which aircraft, flights, crew members, passengers, cargo and airport resources must be managed efficiently. Every flight requires accurate scheduling, aircraft assignment, crew allocation, cargo handling and monitoring. A small error in any one of these areas can result in delays, increased operational costs and disruption of the overall transportation network.

Traditional or fragmented information systems often maintain operational data in separate modules. Such systems can make it difficult to obtain a unified view of the current status of flights and logistics operations. Problems such as flight delays, inappropriate crew assignment, cargo-handling issues and inefficient fuel usage may therefore become difficult to monitor and analyse.

The proposed **Modern Airline Logistics and Sustainability Management System** provides a centralized database for airline logistics operations. The system manages aircraft information, flight schedules, crew assignments, passenger information, loyalty-based baggage allowances, specialized cargo, carbon emissions and operational disruptions.

The system gives special attention to three modern airline challenges:

1. **Cold-Chain Cargo Management** – monitoring temperature-sensitive medical and perishable cargo.
2. **Sustainability Management** – recording fuel consumption and carbon dioxide emissions for each flight.
3. **Disruption Management** – recording and analysing delays caused by weather, technical problems, ATC, crew, security and other operational events.

BACKGROUND OF THE PROJECT WORK

Airline operations have traditionally depended on multiple independent systems to manage flight schedules, fleet details, passenger information, crew assignments and cargo handling. As airline operations expanded, the amount of data generated by each flight increased significantly. Separate systems and manual records can result in duplication, inconsistency and difficulty in obtaining a complete view of airline logistics.

Modern air transportation also faces challenges that go beyond basic passenger scheduling. Airlines transport temperature-sensitive pharmaceutical products, food, biological samples and other specialized cargo that require controlled environmental conditions during transportation. Therefore, cargo information must include handling requirements and temperature-monitoring information.

Environmental sustainability has also become an important part of airline operations. Fuel consumption and carbon emissions must be monitored so that organizations can analyse the environmental impact of individual flights and routes.

Another major challenge is operational disruption. Flights may be delayed because of weather conditions, technical faults, air-traffic restrictions, crew availability or security-related problems. Recording these events in a structured database allows management to identify frequent causes of delay and analyse operational performance.

The proposed database applies **Entity-Relationship modelling and relational normalization principles** to create a centralized airline logistics system. The design uses primary and foreign keys to maintain relationships among entities and uses constraints to prevent invalid data from entering the database.

HARDWARE AND SOFTWARE REQUIREMENTS

- **Hardware Requirements:**

Component	Minimum Requirement	Recommended
Processor	Intel Core i3 / equivalent	Intel Core i5 or higher
RAM	4 GB	8 GB or higher
Storage	20 GB free space	256 GB SSD or higher
Display	1366 × 768	1920 × 1080
Input	Keyboard and mouse	Keyboard and mouse
Network	Wi-Fi/Ethernet	Stable broadband/LAN

- **Software Requirements:**

Component	Specification
Operating System	Windows 10/11, Linux or macOS
DBMS	MySQL 8.0
Query Interface	MySQL Workbench / MySQL Command Line Client
Query Language	SQL
ER Diagram Tool	draw.io / Lucidchart
Documentation	Microsoft Word / Google Docs
Browser	Google Chrome / Microsoft Edge

E-R DIAGRAM

Entity	Primary Key (PK)	Attributes	Foreign Key (FK)
AIRCRAFT	Aircraft_ID	Model, Capacity_kg, Fuel_Efficiency, Aircraft_Status	—
FLIGHT	Flight_ID	Route_Code, Dep_Time, Arr_Time, Flight_Status	Aircraft_ID → AIRCRAFT
CREW	Crew_ID	Crew_Name, Role, Base_Location	—
FLIGHT_CREW	Assignment_ID	Shift_Hours	Flight_ID → FLIGHT, Crew_ID → CREW
LOYALTY_TIER	Tier_ID	Tier_Name, Extra_Baggage_kg	—
PASSENGER	Passenger_ID	Passenger_Name, Passport_No, Baggage_kg	Tier_ID → LOYALTY_TIER
SPEC_CARGO	Cargo_ID	Cargo_Description, Weight_kg, Category, Req_Temp_C, Current_Temp_C, Tracking_Status	Flight_ID → FLIGHT
EMISSIONS	Log_ID	Fuel_Liters, CO2_Tons, Logged_At	Flight_ID → FLIGHT
DISRUPTION	Event_ID	Delay_Min, Reason_Code, Event_Date, Event_Status	Flight_ID → FLIGHT

DESCRIPTION ABOUT DESIGN OF THE DATABASE

The database is designed using relational database principles with emphasis on integrity, consistency and reduction of redundancy. The schema contains nine interconnected entities representing major airline logistics operations.

The **FLIGHT** entity acts as the central operational entity. Each flight is associated with one valid aircraft through the Aircraft_ID foreign key. This ensures that a flight cannot be scheduled without an aircraft existing in the database.

The relationship between **FLIGHT and CREW** is many-to-many because one flight requires multiple crew members and one crew member may participate in multiple flights. This relationship is resolved through the FLIGHT_CREW associative entity.

The **LOYALTY_TIER** and **PASSENGER** relationship allows the system to determine additional baggage allowances based on passenger membership level.

The **SPEC_CARGO** entity stores information about specialized cargo associated with flights. The design supports medical, perishable, hazardous and standard cargo and allows

temperature-sensitive shipments to be monitored using required and current temperature values.

The **EMISSIONS** entity stores fuel consumption and carbon dioxide emission information for flights. This enables the airline to analyse environmental impact at flight and route level.

The **DISRUPTION** entity stores operational interruptions, including delay duration, reason and status. This supports analysis of delay patterns and operational performance.

Constraints such as PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK and DEFAULT have been applied to maintain data validity. The design follows normalization principles so that information is stored in logically separated relations.

Relationships / Cardinality Table

Parent Entity	Relationship	Child Entity	Cardinality	Meaning
AIRCRAFT	assigned to	FLIGHT	1 : M	One aircraft can operate many flights over time; each flight uses one aircraft.
FLIGHT	assigned through	FLIGHT_CREW	1 : M	One flight can have multiple crew assignments.
CREW	assigned through	FLIGHT_CREW	1 : M	One crew member can be assigned to multiple flights.
FLIGHT	carries	SPEC_CARGO	1 : M	One flight can transport multiple cargo shipments.
FLIGHT	generates	EMISSIONS	1 : M	A flight can have one or more emission logs.
FLIGHT	experiences	DISRUPTION	1 : M	One flight can experience multiple disruption events.
LOYALTY_TIER	assigned to	PASSENGER	1 : M	One loyalty tier can contain many passengers.
FLIGHT	has	CREW	M : N	A flight can have many crew members and a crew member can work on many flights; resolved through FLIGHT_CREW.

RELATIONAL SCHEMA

AIRCRAFT(
  Aircraft_ID PK,
  Model,
  Capacity_kg,
  Fuel_Efficiency,
  Aircraft_Status
)

FLIGHT(
  Flight_ID PK,
  Route_Code,
  Dep_Time,
  Arr_Time,
  Aircraft_ID FK → AIRCRAFT(Aircraft_ID),
  Flight_Status
)

CREW(
  Crew_ID PK,
  Crew_Name,
  Role,
  Base_Location
)

FLIGHT_CREW(
  Assignment_ID PK,
  Flight_ID FK → FLIGHT(Flight_ID),
  Crew_ID FK → CREW(Crew_ID),
  Shift_Hours
)

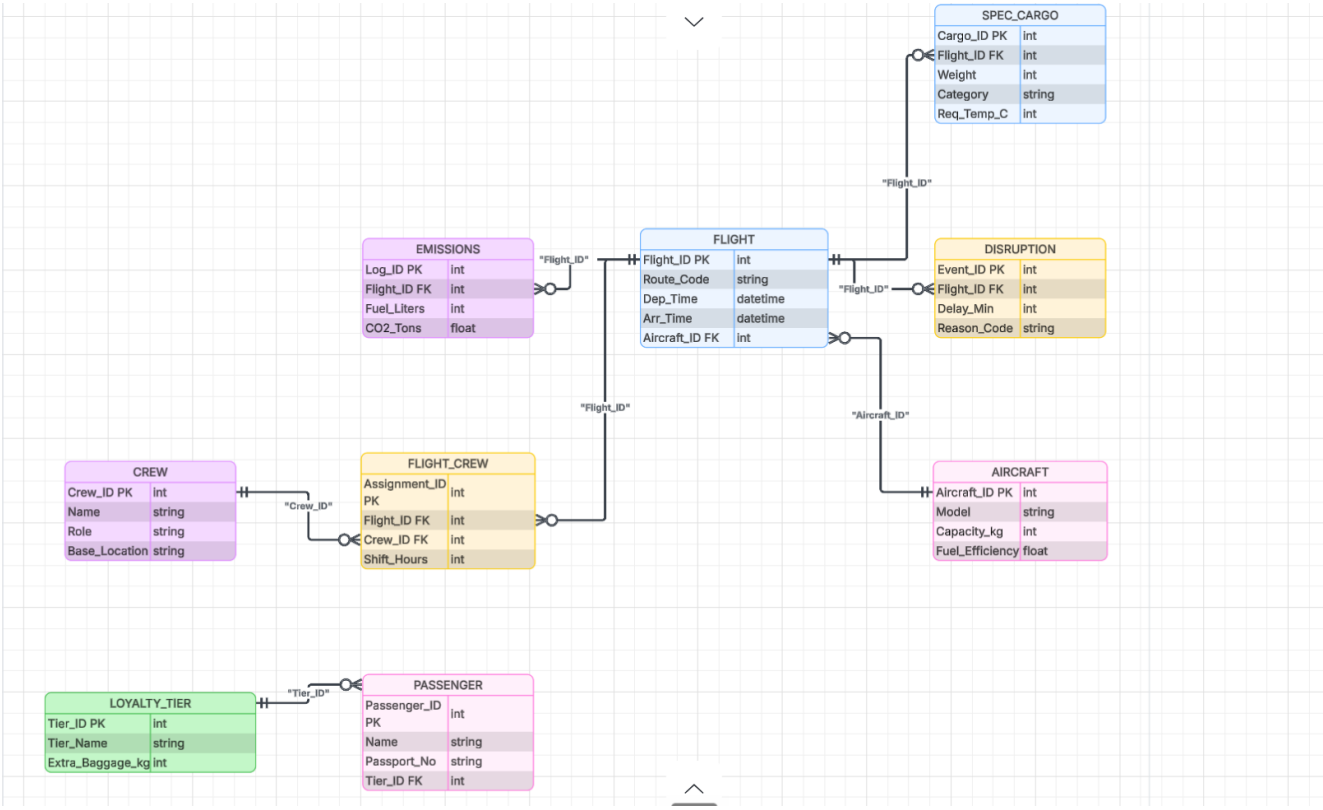
LOYALTY_TIER(
  Tier_ID PK,
  Tier_Name UNIQUE,
  Extra_Baggage_kg
)

PASSENGER(
Passenger_ID PK,
Passenger_Name,
Passport_No UNIQUE,
Tier_ID FK → LOYALTY_TIER(Tier_ID),
Baggage_kg
)

SPEC_CARGO(
Cargo_ID PK,
Flight_ID FK → FLIGHT(Flight_ID),
Cargo_Description,
Weight_kg,
Category,
Req_Temp_C,
Current_Temp_C,
Tracking_Status
)

EMISSIONS(
Log_ID PK,
Flight_ID FK → FLIGHT(Flight_ID),
Fuel_Liters,
CO2_Tons,
Logged_At
)

DISRUPTION(
Event_ID PK,
Flight_ID FK → FLIGHT(Flight_ID),
Delay_Min,
Reason_Code,
Event_Date,
Event_Status
)


SPEC_CARGO
Cargo_ID PK int
Flight_ID FK int
Weight int
Category string
Req_Temp_C int
"Flight_ID"
EMISSIONS
Log_ID PK int
Flight_ID FK int
Fuel_Liters int
CO2_Tons float
"Flight_ID"
FLIGHT
Flight_ID PK int
Route_Code string
Dep_Time datetime
Arr_Time datetime
Aircraft_ID FK int
"Flight_ID"
DISRUPTION
Event_ID PK int
Flight_ID FK int
Delay_Min int
Reason_Code string
"Flight_ID"
"Aircraft_ID"
CREW
Crew_ID PK int
Name string
Role string
Base_Location string
"Crew_ID"
FLIGHT_CREW
Assignment_ID PK int
Flight_ID FK int
Crew_ID FK int
Shift_Hours int
AIRCRAFT
Aircraft_ID PK int
Model string
Capacity_kg int
Fuel_Efficiency float
LOYALTY_TIER
Tier_ID PK int
Tier_Name string
Extra_Baggage_kg int
"Tier_ID"
PASSENGER
Passenger_ID PK int
Name string
Passport_No string
Tier_ID FK int

EXECUTION OF THE SQL QUERY

1. TABLE CREATION WITH ALL CONSTRAINTS

```
SQL> CREATE TABLE AIRCRAFT (
  2      Aircraft_ID NUMBER(10) PRIMARY KEY,
  3      Model VARCHAR2(50) NOT NULL,
  4      Capacity_kg NUMBER(10,2) NOT NULL,
  5      Fuel_Efficiency NUMBER(8,3) NOT NULL,
  6      Aircraft_Status VARCHAR2(20) DEFAULT 'ACTIVE' NOT NULL,
  7      CONSTRAINT chk_aircraft_capacity CHECK (Capacity_kg > 0),
  8      CONSTRAINT chk_aircraft_efficiency CHECK (Fuel_Efficiency > 0),
  9      CONSTRAINT chk_aircraft_status
 10          CHECK (Aircraft_Status IN ('ACTIVE', 'MAINTENANCE', 'RETIRED'))
 11  );

Table created.
```

```
SQL> CREATE TABLE LOYALTY_TIER (
  2      Tier_ID NUMBER(10) PRIMARY KEY,
  3      Tier_Name VARCHAR2(30) NOT NULL UNIQUE,
  4      Extra_Baggage_kg NUMBER(6,2) DEFAULT 0 NOT NULL,
  5      CONSTRAINT chk_tier_baggage CHECK (Extra_Baggage_kg >= 0)
  6  );

Table created.
```

```
SQL> CREATE TABLE CREW (
  2      Crew_ID NUMBER(10) PRIMARY KEY,
  3      Crew_Name VARCHAR2(60) NOT NULL,
  4      Role VARCHAR2(30) NOT NULL,
  5      Base_Location VARCHAR2(50) NOT NULL,
  6      CONSTRAINT chk_crew_role
  7          CHECK (Role IN ('PILOT', 'CO-PILOT', 'CABIN CREW', 'ENGINEER',
'GROUND STAFF'))
  8  );

Table created.
```

```
SQL> CREATE TABLE PASSENGER (
  2      Passenger_ID NUMBER(10) PRIMARY KEY,
  3      Passenger_Name VARCHAR2(60) NOT NULL,
  4      Passport_No VARCHAR2(20) NOT NULL UNIQUE,
  5      Tier_ID NUMBER(10) NOT NULL,
  6      Baggage_kg NUMBER(6,2) DEFAULT 0 NOT NULL,
  7      CONSTRAINT fk_passenger_tier
  8          FOREIGN KEY (Tier_ID) REFERENCES LOYALTY_TIER(Tier_ID),
  9      CONSTRAINT chk_passenger_baggage CHECK (Baggage_kg >= 0)
 10  );

Table created.
```

```
SQL> CREATE TABLE FLIGHT (
  2      Flight_ID NUMBER(10) PRIMARY KEY,
  3      Route_Code VARCHAR2(20) NOT NULL,
  4      Dep_Time TIMESTAMP NOT NULL,
  5      Arr_Time TIMESTAMP NOT NULL,
  6      Aircraft_ID NUMBER(10) NOT NULL,
  7      Flight_Status VARCHAR2(20) DEFAULT 'SCHEDULED' NOT NULL,
  8      CONSTRAINT fk_flight_aircraft
  9          FOREIGN KEY (Aircraft_ID) REFERENCES AIRCRAFT(Aircraft_ID),
 10      CONSTRAINT chk_flight_time CHECK (Arr_Time > Dep_Time),
 11      CONSTRAINT chk_flight_status
 12          CHECK (Flight_Status IN
 13          ('SCHEDULED', 'DEPARTED', 'ARRIVED', 'DELAYED', 'CANCELLED'))
 14  );

Table created.
```

```
SQL> CREATE TABLE FLIGHT_CREW (
  2      Assignment_ID NUMBER(10) PRIMARY KEY,
  3      Flight_ID NUMBER(10) NOT NULL,
  4      Crew_ID NUMBER(10) NOT NULL,
  5      Shift_Hours NUMBER(5,2) NOT NULL,
  6      CONSTRAINT fk_fc_flight
  7          FOREIGN KEY (Flight_ID) REFERENCES FLIGHT(Flight_ID),
  8      CONSTRAINT fk_fc_crew
  9          FOREIGN KEY (Crew_ID) REFERENCES CREW(Crew_ID),
 10      CONSTRAINT uq_flight_crew UNIQUE (Flight_ID, Crew_ID),
 11      CONSTRAINT chk_shift_hours CHECK (Shift_Hours > 0)
 12  );

Table created.
```

```
SQL> CREATE TABLE SPEC_CARGO (
  2      Cargo_ID NUMBER(10) PRIMARY KEY,
  3      Flight_ID NUMBER(10) NOT NULL,
  4      Cargo_Description VARCHAR2(100) NOT NULL,
  5      Weight_kg NUMBER(10,2) NOT NULL,
  6      Category VARCHAR2(20) NOT NULL,
  7      Req_Temp_C NUMBER(5,2),
  8      Current_Temp_C NUMBER(5,2),
  9      Tracking_Status VARCHAR2(20) DEFAULT 'IN TRANSIT' NOT NULL,
 10      CONSTRAINT fk_cargo_flight
 11          FOREIGN KEY (Flight_ID) REFERENCES FLIGHT(Flight_ID),
 12      CONSTRAINT chk_cargo_weight CHECK (Weight_kg > 0),
 13      CONSTRAINT chk_cargo_category
 14          CHECK (Category IN ('MEDICAL', 'PERISHABLE', 'HAZARDOUS', 'STANDARD')),
 15      CONSTRAINT chk_tracking_status
 16          CHECK (Tracking_Status IN ('RECEIVED', 'IN TRANSIT', 'DELIVERED', 'HOLD'))
 17  );

Table created.
```

```
SQL> CREATE TABLE EMISSIONS (
  2      Log_ID NUMBER(10) PRIMARY KEY,
  3      Flight_ID NUMBER(10) NOT NULL,
  4      Fuel_Liters NUMBER(12,2) NOT NULL,
  5      CO2_Tons NUMBER(10,3) NOT NULL,
  6      Logged_At TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
  7      CONSTRAINT fk_emission_flight
  8          FOREIGN KEY (Flight_ID) REFERENCES FLIGHT(Flight_ID),
  9      CONSTRAINT chk_fuel CHECK (Fuel_Liters > 0),
 10      CONSTRAINT chk_co2 CHECK (CO2_Tons >= 0)
 11  );

Table created.
```

```
SQL> CREATE TABLE DISRUPTION (
  2      Event_ID NUMBER(10) PRIMARY KEY,
  3      Flight_ID NUMBER(10) NOT NULL,
  4      Delay_Min NUMBER(6) NOT NULL,
  5      Reason_Code VARCHAR2(20) NOT NULL,
  6      Event_Date TIMESTAMP NOT NULL,
  7      Event_Status VARCHAR2(20) DEFAULT 'OPEN' NOT NULL,
  8      CONSTRAINT fk_disruption_flight
  9          FOREIGN KEY (Flight_ID) REFERENCES FLIGHT(Flight_ID),
 10      CONSTRAINT chk_delay CHECK (Delay_Min >= 0),
 11      CONSTRAINT chk_reason_code
 12          CHECK (Reason_Code IN
 13          ('WEATHER', 'TECHNICAL', 'ATC', 'CREW', 'SECURITY', 'OTHER')),
 14      CONSTRAINT chk_event_status
 15          CHECK (Event_Status IN ('OPEN', 'RESOLVED'))
 16  );

Table created.
```

2. INSERTION - 5 ROWS PER TABLE

Aircraft table:

```
SQL> INSERT INTO AIRCRAFT
  2  (Aircraft_ID, Model, Capacity_kg, Fuel_Efficiency, Aircraft_Status)
  3  VALUES (101, 'Airbus A320', 16800, 3.10, 'ACTIVE');

1 row created.

SQL> INSERT INTO AIRCRAFT
  2  (Aircraft_ID, Model, Capacity_kg, Fuel_Efficiency, Aircraft_Status)
  3  VALUES (102, 'Boeing 737-800', 20800, 2.95, 'ACTIVE');

1 row created.

SQL>
SQL>
SQL> INSERT INTO AIRCRAFT
  2  (Aircraft_ID, Model, Capacity_kg, Fuel_Efficiency, Aircraft_Status)
  3  VALUES (103, 'Airbus A321', 24000, 3.25, 'ACTIVE');

1 row created.

SQL> INSERT INTO AIRCRAFT
  2  (Aircraft_ID, Model, Capacity_kg, Fuel_Efficiency, Aircraft_Status)
  3  VALUES (104, 'Boeing 787-9', 37000, 2.40, 'ACTIVE');

1 row created.
```

```
SQL> INSERT INTO AIRCRAFT
  2  (Aircraft_ID, Model, Capacity_kg, Fuel_Efficiency, Aircraft_Status)
  3  VALUES (105, 'Airbus A330-300', 45000, 2.20, 'MAINTENANCE');

1 row created.
```

Loyalty_tier table:

```
SQL> INSERT INTO LOYALTY_TIER
  2  (Tier_ID, Tier_Name, Extra_Baggage_kg)
  3  VALUES (1, 'SILVER', 5);

1 row created.

SQL> INSERT INTO LOYALTY_TIER
  2  (Tier_ID, Tier_Name, Extra_Baggage_kg)
  3  VALUES (2, 'GOLD', 10);

1 row created.

SQL> INSERT INTO LOYALTY_TIER
  2  (Tier_ID, Tier_Name, Extra_Baggage_kg)
  3  VALUES (3, 'PLATINUM', 15);

1 row created.

SQL> INSERT INTO LOYALTY_TIER
  2  (Tier_ID, Tier_Name, Extra_Baggage_kg)
  3  VALUES (4, 'DIAMOND', 25);

1 row created.
```

```
SQL> INSERT INTO LOYALTY_TIER
  2  (Tier_ID, Tier_Name, Extra_Baggage_kg)
  3  VALUES (5, 'BASIC', 0);

1 row created.
```

Crew table:

```
SQL> INSERT INTO CREW
  2  (Crew_ID, Crew_Name, Role, Base_Location)
  3  VALUES (201, 'Ananya Sharma', 'PILOT', 'DELHI');

1 row created.

SQL>
SQL> INSERT INTO CREW
  2  (Crew_ID, Crew_Name, Role, Base_Location)
  3  VALUES (202, 'Rahul Mehta', 'CO-PILOT', 'MUMBAI');

1 row created.

SQL>
SQL> INSERT INTO CREW
  2  (Crew_ID, Crew_Name, Role, Base_Location)
  3  VALUES (203, 'Ishita Rao', 'CABIN CREW', 'BENGALURU');

1 row created.

SQL>
SQL> INSERT INTO CREW
  2  (Crew_ID, Crew_Name, Role, Base_Location)
  3  VALUES (204, 'Vikram Singh', 'ENGINEER', 'DELHI');

1 row created.

SQL>
SQL> INSERT INTO CREW
  2  (Crew_ID, Crew_Name, Role, Base_Location)
  3  VALUES (205, 'Neha Kapoor', 'CABIN CREW', 'HYDERABAD');

1 row created.
```

Flight table:

```
SQL> INSERT INTO FLIGHT
  2  (Flight_ID, Route_Code, Dep_Time, Arr_Time, Aircraft_ID, Flight_Status)
  3  VALUES (
  4  301, 'DEL-MUM',
  5  TIMESTAMP '2026-08-20 06:30:00',
  6  TIMESTAMP '2026-08-20 08:45:00',
  7  101, 'SCHEDULED');

1 row created.

SQL>
SQL> INSERT INTO FLIGHT
  2  (Flight_ID, Route_Code, Dep_Time, Arr_Time, Aircraft_ID, Flight_Status)
  3  VALUES (
  4  302, 'MUM-BLR',
  5  TIMESTAMP '2026-08-20 10:00:00',
  6  TIMESTAMP '2026-08-20 12:00:00',
  7  102, 'SCHEDULED');

1 row created.

SQL>
SQL> INSERT INTO FLIGHT
  2  (Flight_ID, Route_Code, Dep_Time, Arr_Time, Aircraft_ID, Flight_Status)
  3  VALUES (
  4  303, 'BLR-DEL',
  5  TIMESTAMP '2026-08-20 14:30:00',
  6  TIMESTAMP '2026-08-20 17:15:00',
  7  103, 'SCHEDULED');

1 row created.

SQL>
SQL> INSERT INTO FLIGHT
  2  (Flight_ID, Route_Code, Dep_Time, Arr_Time, Aircraft_ID, Flight_Status)
  3  VALUES (
  4  304, 'DEL-DXB',
  5  TIMESTAMP '2026-08-21 22:00:00',
  6  TIMESTAMP '2026-08-22 00:30:00',
  7  104, 'SCHEDULED');

1 row created.
```

```
SQL> INSERT INTO FLIGHT
  2  (Flight_ID, Route_Code, Dep_Time, Arr_Time, Aircraft_ID, Flight_Status)
  3  VALUES (
  4  305, 'MUM-SIN',
  5  TIMESTAMP '2026-08-21 09:00:00',
  6  TIMESTAMP '2026-08-21 17:00:00',
  7  105, 'DELAYED');

1 row created.
```

Flight_crew table:

```
SQL> INSERT INTO FLIGHT_CREW
  2  (Assignment_ID, Flight_ID, Crew_ID, Shift_Hours)
  3  VALUES (401, 301, 201, 4.00);

1 row created.

SQL>
SQL> INSERT INTO FLIGHT_CREW
  2  (Assignment_ID, Flight_ID, Crew_ID, Shift_Hours)
  3  VALUES (402, 301, 203, 4.00);

1 row created.

SQL>
SQL> INSERT INTO FLIGHT_CREW
  2  (Assignment_ID, Flight_ID, Crew_ID, Shift_Hours)
  3  VALUES (403, 302, 202, 3.50);

1 row created.

SQL>
SQL> INSERT INTO FLIGHT_CREW
  2  (Assignment_ID, Flight_ID, Crew_ID, Shift_Hours)
  3  VALUES (404, 303, 204, 5.00);

1 row created.

SQL>
SQL> INSERT INTO FLIGHT_CREW
  2  (Assignment_ID, Flight_ID, Crew_ID, Shift_Hours)
  3  VALUES (405, 304, 205, 6.00);

1 row created.
```

Passenger table:

```
SQL> INSERT INTO PASSENGER
  2  (Passenger_ID, Passenger_Name, Passport_No, Tier_ID, Baggage_kg)
  3  VALUES (501, 'Aarav Patel', 'P1234501', 2, 28);

1 row created.

SQL>
SQL> INSERT INTO PASSENGER
  2  (Passenger_ID, Passenger_Name, Passport_No, Tier_ID, Baggage_kg)
  3  VALUES (502, 'Meera Nair', 'P1234502', 3, 35);

1 row created.

SQL>
SQL> INSERT INTO PASSENGER
  2  (Passenger_ID, Passenger_Name, Passport_No, Tier_ID, Baggage_kg)
  3  VALUES (503, 'Rohan Verma', 'P1234503', 5, 20);

1 row created.

SQL>
SQL> INSERT INTO PASSENGER
  2  (Passenger_ID, Passenger_Name, Passport_No, Tier_ID, Baggage_kg)
  3  VALUES (504, 'Sara Khan', 'P1234504', 1, 25);

1 row created.

SQL>
SQL> INSERT INTO PASSENGER
  2  (Passenger_ID, Passenger_Name, Passport_No, Tier_ID, Baggage_kg)
  3  VALUES (505, 'Kabir Joshi', 'P1234505', 4, 42);

1 row created.
```

Spec_cargo table:

```
SQL> INSERT INTO SPEC_CARGO
  2  (Cargo_ID, Flight_ID, Cargo_Description, Weight_kg, Category,
  3  Req_Temp_C, Current_Temp_C, Tracking_Status)
  4  VALUES (601, 301, 'COVID Vaccine Shipment', 1200, 'MEDICAL',
  5  4, 4.20, 'IN TRANSIT');

1 row created.

SQL>
SQL> INSERT INTO SPEC_CARGO
  2  (Cargo_ID, Flight_ID, Cargo_Description, Weight_kg, Category,
  3  Req_Temp_C, Current_Temp_C, Tracking_Status)
  4  VALUES (602, 302, 'Fresh Seafood', 850, 'PERISHABLE',
  5  2, 2.40, 'IN TRANSIT');

1 row created.

SQL>
SQL> INSERT INTO SPEC_CARGO
  2  (Cargo_ID, Flight_ID, Cargo_Description, Weight_kg, Category,
  3  Req_Temp_C, Current_Temp_C, Tracking_Status)
  4  VALUES (603, 303, 'Industrial Chemicals', 1500, 'HAZARDOUS',
  5  NULL, NULL, 'RECEIVED');

1 row created.

SQL>
SQL> INSERT INTO SPEC_CARGO
  2  (Cargo_ID, Flight_ID, Cargo_Description, Weight_kg, Category,
  3  Req_Temp_C, Current_Temp_C, Tracking_Status)
  4  VALUES (604, 304, 'Fresh Produce', 2100, 'PERISHABLE',
  5  5, 5.10, 'IN TRANSIT');

1 row created.

SQL>
SQL> INSERT INTO SPEC_CARGO
  2  (Cargo_ID, Flight_ID, Cargo_Description, Weight_kg, Category,
  3  Req_Temp_C, Current_Temp_C, Tracking_Status)
  4  VALUES (605, 305, 'Electronic Equipment', 900, 'STANDARD',
  5  NULL, NULL, 'DELIVERED');

1 row created.
```

Emissions table:

```
SQL> INSERT INTO EMISSIONS
  2  (Log_ID, Flight_ID, Fuel_Liters, CO2_Tons, Logged_At)
  3  VALUES (701, 301, 4200, 13.020,
  4  TIMESTAMP '2026-08-20 09:00:00');

1 row created.

SQL>
SQL> INSERT INTO EMISSIONS
  2  (Log_ID, Flight_ID, Fuel_Liters, CO2_Tons, Logged_At)
  3  VALUES (702, 302, 3800, 11.210,
  4  TIMESTAMP '2026-08-20 12:30:00');

1 row created.

SQL>
SQL> INSERT INTO EMISSIONS
  2  (Log_ID, Flight_ID, Fuel_Liters, CO2_Tons, Logged_At)
  3  VALUES (703, 303, 4600, 14.950,
  4  TIMESTAMP '2026-08-20 17:45:00');

1 row created.

SQL>
SQL> INSERT INTO EMISSIONS
  2  (Log_ID, Flight_ID, Fuel_Liters, CO2_Tons, Logged_At)
  3  VALUES (704, 304, 12500, 30.000,
  4  TIMESTAMP '2026-08-22 01:00:00');

1 row created.

SQL>
SQL> INSERT INTO EMISSIONS
  2  (Log_ID, Flight_ID, Fuel_Liters, CO2_Tons, Logged_At)
  3  VALUES (705, 305, 14000, 31.500,
  4  TIMESTAMP '2026-08-21 18:00:00');

1 row created.
```

Disruption table:

```
SQL> INSERT INTO DISRUPTION
  2  (Event_ID, Flight_ID, Delay_Min, Reason_Code, Event_Date, Event_Status)
  3  VALUES (801, 301, 20, 'ATC',
  4  TIMESTAMP '2026-08-20 06:50:00', 'RESOLVED');

1 row created.

SQL>
SQL> INSERT INTO DISRUPTION
  2  (Event_ID, Flight_ID, Delay_Min, Reason_Code, Event_Date, Event_Status)
  3  VALUES (802, 302, 35, 'WEATHER',
  4  TIMESTAMP '2026-08-20 10:20:00', 'RESOLVED');

1 row created.

SQL>
SQL> INSERT INTO DISRUPTION
  2  (Event_ID, Flight_ID, Delay_Min, Reason_Code, Event_Date, Event_Status)
  3  VALUES (803, 303, 15, 'CREW',
  4  TIMESTAMP '2026-08-20 14:40:00', 'RESOLVED');

1 row created.

SQL>
SQL> INSERT INTO DISRUPTION
  2  (Event_ID, Flight_ID, Delay_Min, Reason_Code, Event_Date, Event_Status)
  3  VALUES (804, 304, 60, 'SECURITY',
  4  TIMESTAMP '2026-08-21 22:30:00', 'OPEN');

1 row created.

SQL>
SQL> INSERT INTO DISRUPTION
  2  (Event_ID, Flight_ID, Delay_Min, Reason_Code, Event_Date, Event_Status)
  3  VALUES (805, 305, 90, 'TECHNICAL',
  4  TIMESTAMP '2026-08-21 08:30:00', 'OPEN');

1 row created.
```

3.DELETION DEMONSTRATION:

Aircraft table:

```
SQL> INSERT INTO AIRCRAFT
  2  (Aircraft_ID, Model, Capacity_kg, Fuel_Efficiency, Aircraft_Status)
  3  VALUES (999, 'Test Aircraft', 10000, 2.50, 'ACTIVE');

1 row created.

SQL> DELETE FROM AIRCRAFT WHERE Aircraft_ID = 999;

1 row deleted.
```

Loyalty_tier table:

```
SQL> INSERT INTO LOYALTY_TIER
  2  (Tier_ID, Tier_Name, Extra_Baggage_kg)
  3  VALUES (999, 'TEST_TIER', 0);

1 row created.

SQL>
SQL> DELETE FROM LOYALTY_TIER WHERE Tier_ID = 999;

1 row deleted.
```

Crew table:

```
SQL> INSERT INTO CREW
  2  (Crew_ID, Crew_Name, Role, Base_Location)
  3  VALUES (999, 'Test Crew', 'CABIN CREW', 'DELHI');

1 row created.

SQL>
SQL> DELETE FROM CREW WHERE Crew_ID = 999;

1 row deleted.
```

Flight table:

```
SQL> INSERT INTO FLIGHT
  2  (Flight_ID, Route_Code, Dep_Time, Arr_Time, Aircraft_ID, Flight_Status)

  3  VALUES (999, 'TEST-TEST',
  4  TIMESTAMP '2026-08-25 08:00:00',
  5  TIMESTAMP '2026-08-25 10:00:00',
  6  101, 'SCHEDULED');

1 row created.

SQL>
SQL> DELETE FROM FLIGHT WHERE Flight_ID = 999;

1 row deleted.
```

Flight_crew table:

```
SQL> INSERT INTO FLIGHT_CREW
  2  (Assignment_ID, Flight_ID, Crew_ID, Shift_Hours)
  3  VALUES (999, 301, 205, 2.00);

1 row created.

SQL>
SQL> DELETE FROM FLIGHT_CREW WHERE Assignment_ID = 999;

1 row deleted.
```

Passenger table:

```
SQL> INSERT INTO PASSENGER
  2  (Passenger_ID, Passenger_Name, Passport_No, Tier_ID, Baggage_kg)
  3  VALUES (999, 'Test Passenger', 'P9999999', 5, 10);

1 row created.

SQL>
SQL> DELETE FROM PASSENGER WHERE Passenger_ID = 999;

1 row deleted.
```

Spec_cargo table:

```
SQL> INSERT INTO SPEC_CARGO
  2  (Cargo_ID, Flight_ID, Cargo_Description, Weight_kg, Category,
  3  Req_Temp_C, Current_Temp_C, Tracking_Status)
  4  VALUES (999, 301, 'Test Cargo', 100, 'STANDARD',
  5  NULL, NULL, 'RECEIVED');

1 row created.

SQL>
SQL> DELETE FROM SPEC_CARGO WHERE Cargo_ID = 999;

1 row deleted.
```

Emissions table:

```
SQL> INSERT INTO EMISSIONS
  2  (Log_ID, Flight_ID, Fuel_Liters, CO2_Tons, Logged_At)
  3  VALUES (999, 301, 1000, 3.00,
  4  TIMESTAMP '2026-08-20 11:00:00');

1 row created.

SQL>
SQL> DELETE FROM EMISSIONS WHERE Log_ID = 999;

1 row deleted.
```

Disruption table:

```
SQL> INSERT INTO DISRUPTION
  2  (Event_ID, Flight_ID, Delay_Min, Reason_Code, Event_Date, Event_Status)

  3  VALUES (999, 301, 10, 'OTHER',
  4  TIMESTAMP '2026-08-20 11:30:00', 'RESOLVED');

1 row created.

SQL>
SQL> DELETE FROM DISRUPTION WHERE Event_ID = 999;

1 row deleted.
```

4.UPDATION - ONE UPDATE ON EACH TABLE

Aircraft table:

```
SQL> UPDATE AIRCRAFT
  2  SET Fuel_Efficiency = 3.20
  3  WHERE Aircraft_ID = 101;

1 row updated.
```

Loyalty_tier table:

```
SQL> UPDATE LOYALTY_TIER
  2  SET Extra_Baggage_kg = 12
  3  WHERE Tier_ID = 2;

1 row updated.
```

Crew table:

```
SQL> UPDATE CREW
  2  SET Base_Location = 'MUMBAI'
  3  WHERE Crew_ID = 202;

1 row updated.
```

Flight table:

```
SQL> UPDATE FLIGHT
  2  SET Flight_Status = 'DELAYED'
  3  WHERE Flight_ID = 301;

1 row updated.
```

Flight_crew table:

```
SQL> UPDATE FLIGHT_CREW
  2  SET Shift_Hours = 4.50
  3  WHERE Assignment_ID = 401;

1 row updated.
```

Passenger table:

```
SQL> UPDATE PASSENGER
  2  SET Baggage_kg = 30
  3  WHERE Passenger_ID = 501;

1 row updated.
```

Spec_cargo table:

```
SQL> UPDATE SPEC_CARGO
  2  SET Current_Temp_C = 4.00
  3  WHERE Cargo_ID = 601;

1 row updated.
```

Emissions table:

```
SQL> UPDATE EMISSIONS
  2  SET CO2_Tons = 13.150
  3  WHERE Log_ID = 701;

1 row updated.
```

Disruption table:

```
SQL> UPDATE DISRUPTION
  2  SET Event_Status = 'RESOLVED'
  3  WHERE Event_ID = 804;

1 row updated.
```

5.SELECT QUERIES

Flight table:

```
SQL> select * from flight;

 FLIGHT_ID ROUTE_CODE          DEP_TIME                                                                    ARR_TIME
         AIRCRAFT_ID FLIGHT_STATUS
---------- ------------------- ------------------------------------------------------------------------- -------------------------------
-------------------------------------- ---------- --------------------
       301 DEL-MUM             20-AUG-26 06.30.00.000000 AM                                              20-AUG-26 08.45.00.000000 AM
           101 DELAYED
       302 MUM-BLR             20-AUG-26 10.00.00.000000 AM                                              20-AUG-26 12.00.00.000000 PM
           102 SCHEDULED
       303 BLR-DEL             20-AUG-26 02.30.00.000000 PM                                              20-AUG-26 05.15.00.000000 PM
           103 SCHEDULED
       304 DEL-DXB             21-AUG-26 10.00.00.000000 PM                                              22-AUG-26 12.30.00.000000 AM
           104 SCHEDULED
       305 MUM-SIN             21-AUG-26 09.00.00.000000 AM                                              21-AUG-26 05.00.00.000000 PM
           105 DELAYED
```

```
SQL> SELECT Flight_ID, Route_Code, Flight_Status
  2  FROM FLIGHT
  3  WHERE Flight_Status = 'DELAYED';

 FLIGHT_ID ROUTE_CODE           FLIGHT_STATUS
---------- -------------------- --------------------
       301 DEL-MUM              DELAYED
       305 MUM-SIN              DELAYED
```

Passenger table:

```
SQL> SELECT Passenger_ID, Passenger_Name, Passport_No, Baggage_kg
  2  FROM PASSENGER;

PASSENGER_ID PASSENGER_NAME
------------ ------------------------------------------------------------
PASSPORT_NO          BAGGAGE_KG
-------------------- ----------
         501 Aarav Patel
P1234501                     30

         502 Meera Nair
P1234502                     35

         503 Rohan Verma
P1234503                     20


PASSENGER_ID PASSENGER_NAME
------------ ------------------------------------------------------------
PASSPORT_NO          BAGGAGE_KG
-------------------- ----------
         504 Sara Khan
P1234504                     25

         505 Kabir Joshi
P1234505                     42
```

6.SINGLE-ROW FUNCTIONS

Aircraft table:

```
SQL> SELECT Aircraft_ID,
  2         Model,
  3         LENGTH(Model) AS Model_Length
  4  FROM AIRCRAFT;

AIRCRAFT_ID MODEL                                              MODEL_LENGTH
----------- -------------------------------------------------- ------------
        101 Airbus A320                                                11
        102 Boeing 737-800                                             14
        103 Airbus A321                                                11
        104 Boeing 787-9                                               12
        105 Airbus A330-300                                            15
```

Crew table:

```
SQL> SELECT Crew_ID,
  2         LOWER(Crew_Name) AS Crew_Name_Lower
  3  FROM CREW;

   CREW_ID CREW_NAME_LOWER
---------- --------------------------------------------------
       201 ananya sharma
       202 rahul mehta
       203 ishita rao
       204 vikram singh
       205 neha kapoor
```

Passenger table:

```
SQL> SELECT Passenger_ID,
  2         UPPER(Passenger_Name) AS Passenger_Name_Upper
  3  FROM PASSENGER;

PASSENGER_ID PASSENGER_NAME_UPPER
------------ --------------------------------------------------
         501 AARAV PATEL
         502 MEERA NAIR
         503 ROHAN VERMA
         504 SARA KHAN
         505 KABIR JOSHI
```

```
SQL> SELECT Passenger_ID,
  2         Passenger_Name || ' - ' || Passport_No AS Passenger_Details
  3  FROM PASSENGER;

PASSENGER_ID PASSENGER_DETAILS
------------ --------------------------------------------------------------------------------
         501 Aarav Patel - P1234501
         502 Meera Nair - P1234502
         503 Rohan Verma - P1234503
         504 Sara Khan - P1234504
         505 Kabir Joshi - P1234505
```

Spec_cargo table:

```
SQL> SELECT Cargo_ID,
  2         Cargo_Description,
  3         COALESCE(Current_Temp_C, 0) AS Display_Temperature
  4  FROM SPEC_CARGO;

  CARGO_ID CARGO_DESCRIPTION                                       DISPLAY_TEMPERATURE
---------- --------------------------------------------------------------------------------
       601 COVID Vaccine Shipment                                                     4
       602 Fresh Seafood                                                            2.4
       603 Industrial Chemicals                                                       0
       604 Fresh Produce                                                            5.1
       605 Electronic Equipment                                                       0
```

Emissions table:

```
SQL> SELECT Flight_ID,
  2         ROUND(CO2_Tons, 2) AS Rounded_CO2_Tons
  3  FROM EMISSIONS;

 FLIGHT_ID ROUNDED_CO2_TONS
---------- ----------------
       301            13.15
       302            11.21
       303            14.95
       304               30
       305             31.5
```

7.AGGREGATE FUNCTIONS

Flight table:

```
SQL> SELECT COUNT(*) AS Total_Flights
  2  FROM FLIGHT;

TOTAL_FLIGHTS
-------------
            5
```

Spec_cargo table:

```
SQL> SELECT SUM(Weight_kg) AS Total_Cargo_Weight
  2  FROM SPEC_CARGO;

TOTAL_CARGO_WEIGHT
------------------
              6550
```

Emissions table:

```
SQL> SELECT MAX(CO2_Tons) AS Maximum_CO2_Tons
  2  FROM EMISSIONS;

MAXIMUM_CO2_TONS
----------------
            31.5
```

```
SQL> SELECT MIN(Fuel_Liters) AS Minimum_Fuel_Used
  2  FROM EMISSIONS;

MINIMUM_FUEL_USED
-----------------
             3800
```

Disruption table:

```
SQL> SELECT ROUND(AVG(Delay_Min), 2) AS Average_Delay_Minutes
  2  FROM DISRUPTION;

AVERAGE_DELAY_MINUTES
---------------------
                   44
```

8.GROUP BY:

Flight table:

```
SQL> SELECT Flight_Status,
  2         COUNT(*) AS Flight_Count
  3  FROM FLIGHT
  4  GROUP BY Flight_Status;

FLIGHT_STATUS         FLIGHT_COUNT
--------------------- ------------
DELAYED                          2
SCHEDULED                        3
```

Spec_cargo table:

```
SQL> SELECT Category,
  2         SUM(Weight_kg) AS Total_Weight_kg
  3  FROM SPEC_CARGO
  4  GROUP BY Category;

CATEGORY             TOTAL_WEIGHT_KG
-------------------- ---------------
MEDICAL                         1200
PERISHABLE                      2950
HAZARDOUS                       1500
STANDARD                         900
```

Disruption table:

```
SQL> SELECT Reason_Code,
  2         SUM(Delay_Min) AS Total_Delay_Minutes
  3  FROM DISRUPTION
  4  GROUP BY Reason_Code;

REASON_CODE          TOTAL_DELAY_MINUTES
-------------------- -------------------
ATC                                   20
WEATHER                               35
CREW                                  15
SECURITY                              60
TECHNICAL                             90
```

9.HAVING:

Spec_cargo table:

```
SQL> SELECT Category,
  2         SUM(Weight_kg) AS Total_Weight_kg
  3  FROM SPEC_CARGO
  4  GROUP BY Category
  5  HAVING SUM(Weight_kg) > 1000;

CATEGORY             TOTAL_WEIGHT_KG
-------------------- ---------------
MEDICAL                         1200
PERISHABLE                      2950
HAZARDOUS                       1500
```

Disruption table:

```
SQL> SELECT Reason_Code,
  2         SUM(Delay_Min) AS Total_Delay_Minutes
  3  FROM DISRUPTION
  4  GROUP BY Reason_Code
  5  HAVING SUM(Delay_Min) > 30;

REASON_CODE          TOTAL_DELAY_MINUTES
-------------------- -------------------
WEATHER                               35
SECURITY                              60
TECHNICAL                             90
```

10.SUBQUERIES:

Aircraft table:

```
SQL> SELECT Aircraft_ID, Model, Fuel_Efficiency
  2  FROM AIRCRAFT
  3  WHERE Fuel_Efficiency > (
  4      SELECT AVG(Fuel_Efficiency)
  5      FROM AIRCRAFT
  6  );

AIRCRAFT_ID MODEL                                              FUEL_EFFICIENCY
----------- -------------------------------------------------- ---------------
        101 Airbus A320                                                  3.2
        102 Boeing 737-800                                              2.95
        103 Airbus A321                                                 3.25
```

Passenger table:

```
SQL> SELECT Passenger_ID, Passenger_Name, Baggage_kg
  2  FROM PASSENGER
  3  WHERE Baggage_kg > (
  4      SELECT AVG(Baggage_kg)
  5      FROM PASSENGER
  6  );

PASSENGER_ID PASSENGER_NAME                                         BAGGAGE_KG
------------ ---------------------------------------------------- ----------
         502 Meera Nair                                                     35
         505 Kabir Joshi                                                    42
```

Emissions table:

```
SQL> SELECT Flight_ID, CO2_Tons
  2  FROM EMISSIONS
  3  WHERE CO2_Tons > (
  4      SELECT AVG(CO2_Tons)
  5      FROM EMISSIONS
  6  );

 FLIGHT_ID   CO2_TONS
---------- ----------
       304         30
       305       31.5
```

11.JOIN QUERIES

Aircraft table AND Flight table:

```
SQL> SELECT F.Flight_ID,
  2         F.Route_Code,
  3         A.Model,
  4         A.Capacity_kg
  5  FROM FLIGHT F
  6  JOIN AIRCRAFT A
  7  ON F.Aircraft_ID = A.Aircraft_ID;

 FLIGHT_ID ROUTE_CODE          MODEL                                              CAPACITY_KG
---------- ------------------- -------------------------------------------------- -----------
       301 DEL-MUM             Airbus A320                                              16800
       302 MUM-BLR             Boeing 737-800                                           20800
       303 BLR-DEL             Airbus A321                                              24000
       304 DEL-DXB             Boeing 787-9                                             37000
       305 MUM-SIN             Airbus A330-300                                          45000
```

Loyalty_tier table AND Passenger:

```
SQL> SELECT P.Passenger_Name,
  2         P.Baggage_kg,
  3         L.Tier_Name,
  4         L.Extra_Baggage_kg
  5  FROM PASSENGER P
  6  JOIN LOYALTY_TIER L
  7  ON P.Tier_ID = L.Tier_ID;

PASSENGER_NAME                                             BAGGAGE_KG TIER_NAME                     EXTRA_BAGGAGE_KG
---------------------------------------------------------- ---------- ------------------------------ ----------------
Sara Khan                                                          25 SILVER                                      5
Aarav Patel                                                        30 GOLD                                       12
Meera Nair                                                         35 PLATINUM                                   15
Kabir Joshi                                                        42 DIAMOND                                    25
Rohan Verma                                                        20 BASIC                                       0
```

Crew table AND Flight table AND Flight_crew table:

```
SQL> SELECT F.Flight_ID,
  2         F.Route_Code,
  3         C.Crew_Name,
  4         C.Role,
  5         FC.Shift_Hours
  6  FROM FLIGHT F
  7  JOIN FLIGHT_CREW FC
  8  ON F.Flight_ID = FC.Flight_ID
  9  JOIN CREW C
 10  ON FC.Crew_ID = C.Crew_ID;

 FLIGHT_ID ROUTE_CODE          CREW_NAME                                          ROLE                           SHIFT_HOURS
---------- ------------------- -------------------------------------------------- ------------------------------ -----------
       301 DEL-MUM             Ananya Sharma                                      PILOT                                  4.5
       302 MUM-BLR             Rahul Mehta                                        CO-PILOT                               3.5
       301 DEL-MUM             Ishita Rao                                         CABIN CREW                               4
       303 BLR-DEL             Vikram Singh                                       ENGINEER                                 5
       304 DEL-DXB             Neha Kapoor                                        CABIN CREW                               6
```

Spec_cargo table AND Flight table:

```
SQL> SELECT F.Flight_ID,
  2         F.Route_Code,
  3         S.Cargo_Description,
  4         S.Category,
  5         S.Weight_kg,
  6         S.Current_Temp_C,
  7         S.Tracking_Status
  8  FROM FLIGHT F
  9  JOIN SPEC_CARGO S
 10  ON F.Flight_ID = S.Flight_ID;

 FLIGHT_ID ROUTE_CODE          CARGO_DESCRIPTION                                                                                   CATEGORY
  WEIGHT_KG CURRENT_TEMP_C TRACKING_STATUS
---------- ---------- ---------- --------------------------------------------------------------------------------------------------- ----------
---------- ----------- -------------- --------------------
       301 DEL-MUM           COVID Vaccine Shipment                                                                              MEDICAL   1
200             4 IN TRANSIT
       302 MUM-BLR           Fresh Seafood                                                                                       PERISHABLE
 850           2.4 IN TRANSIT
       303 BLR-DEL           Industrial Chemicals                                                                                HAZARDOUS 1
500             RECEIVED
       304 DEL-DXB           Fresh Produce                                                                                       PERISHABLE
2100            5.1 IN TRANSIT
       305 MUM-SIN           Electronic Equipment                                                                                STANDARD
900             DELIVERED
```

Flight table AND Aircraft table:

```
SQL> SELECT F.Flight_ID,
  2         F.Route_Code,
  3         A.Model,
  4         E.Fuel_Liters,
  5         E.CO2_Tons
  6  FROM FLIGHT F
  7  JOIN AIRCRAFT A
  8  ON F.Aircraft_ID = A.Aircraft_ID
  9  JOIN EMISSIONS E
 10  ON F.Flight_ID = E.Flight_ID;

 FLIGHT_ID ROUTE_CODE          MODEL                                              FUEL_LITERS   CO2_TONS
---------- ------------------- -------------------------------------------------- ----------- ----------
       301 DEL-MUM             Airbus A320                                              4200      13.15
       302 MUM-BLR             Boeing 737-800                                           3800      11.21
       303 BLR-DEL             Airbus A321                                              4600      14.95
       304 DEL-DXB             Boeing 787-9                                            12500         30
       305 MUM-SIN             Airbus A330-300                                         14000       31.5
```

Disruption table AND Flight table:

```
SQL> SELECT F.Route_Code,
  2         D.Delay_Min,
  3         D.Reason_Code,
  4         D.Event_Status
  5  FROM FLIGHT F
  6  JOIN DISRUPTION D
  7  ON F.Flight_ID = D.Flight_ID
  8  ORDER BY D.Delay_Min DESC;

ROUTE_CODE          DELAY_MIN REASON_CODE         EVENT_STATUS
------------------- ---------- ------------------- --------------------
MUM-SIN                     90 TECHNICAL           OPEN
DEL-DXB                     60 SECURITY            RESOLVED
MUM-BLR                     35 WEATHER             RESOLVED
DEL-MUM                     20 ATC                 RESOLVED
BLR-DEL                     15 CREW                RESOLVED
```

FRONT END APPLICATION DESIGN

The front end is a browser-based airline operations workspace implemented in index.html. It uses a persistent navigation sidebar and a content area for module-specific tables and dashboards. The screens use a Northstar Airways identity, with clear status labels, search controls, refresh actions, modal forms and CSV export.

The overview presents flight counts, fleet availability, open disruptions and CO2 totals, with a departure-flow visualization and upcoming departures. The chart is labelled as sample data and should be treated as illustrative unless the underlying aggregation is verified. The displayed values are snapshots of the supplied demonstration database.

Implemented modules

Module	Main information and actions
Overview	Summary cards, departure visualization and upcoming flights.
Flight schedule	Flight ID, route, departure/arrival, aircraft, status; search, filters, add, details, edit, delete and CSV.
Fleet	Aircraft ID/model, capacity, fuel efficiency, today's flight and status.
Crew roster	Crew member, role, base, assignment, shift hours and duty status.
Cargo tracking	Shipment category, flight, weight, required/current temperature and tracking state.
Passengers & loyalty	Passenger ID, loyalty tier and baggage allowance.
Disruptions	Flight, route, cause, delay, event time and response status.
Emissions	Flight-level fuel and CO2 records and summary metrics.

Sign-in and account registration gate access to the workspace. The module screens provide a unified view over the nine database entities. A dedicated FLIGHT_CREW assignment-management page is not visible in the supplied interface; adding one is a future enhancement.

BACK END DATABASE TABLES

This data dictionary supplements the Review 0 relational schema with logical Oracle-friendly types and integrity rules. Exact deployed column widths and constraints should be checked against Oracle metadata before migration. Types below are representative and are not asserted as exact column declarations where the source schema does not record lengths.

Table	Attributes and logical data types	Key and constraints
AIRCRAFT	AIRCRAFT_ID NUMBER (PK); MODEL VARCHAR2; CAPACITY_KG NUMBER; FUEL_EFFICIENCY NUMBER; AIRCRAFT_STATUS VARCHAR2	Primary key; valid capacity/status domains.
FLIGHT	FLIGHT_ID NUMBER (PK); ROUTE_CODE VARCHAR2; DEP_TIME TIMESTAMP; ARR_TIME TIMESTAMP; AIRCRAFT_ID NUMBER; FLIGHT_STATUS VARCHAR2	FK AIRCRAFT_ID → AIRCRAFT; required route/time/status.
CREW	CREW_ID NUMBER (PK); CREW_NAME VARCHAR2; ROLE VARCHAR2; BASE_LOCATION VARCHAR2	Primary key; required crew identity fields.
FLIGHT_CREW	ASSIGNMENT_ID NUMBER (PK); FLIGHT_ID NUMBER; CREW_ID NUMBER; SHIFT_HOURS NUMBER	FKs to FLIGHT and CREW; valid assignment hours.
LOYALTY_TIER	TIER_ID NUMBER (PK); TIER_NAME VARCHAR2; EXTRA_BAGGAGE_KG NUMBER	Unique tier name; nonnegative allowance.
PASSENGER	PASSENGER_ID NUMBER (PK); PASSENGER_NAME VARCHAR2; PASSPORT_NO VARCHAR2; TIER_ID NUMBER; BAGGAGE_KG NUMBER	Unique passport; FK TIER_ID → LOYALTY_TIER.
SPEC_CARGO	CARGO_ID NUMBER (PK); FLIGHT_ID NUMBER; CARGO_DESCRIPTION VARCHAR2; WEIGHT_KG NUMBER; CATEGORY VARCHAR2; REQ_TEMP_C NUMBER; CURRENT_TEMP_C NUMBER; TRACKING_STATUS VARCHAR2	FK FLIGHT_ID; nonnegative weight; temperature fields.
EMISSIONS	LOG_ID NUMBER (PK); FLIGHT_ID NUMBER; FUEL_LITERS NUMBER; CO2_TONS NUMBER; LOGGED_AT TIMESTAMP	FK FLIGHT_ID; nonnegative logged quantities.
DISRUPTION	EVENT_ID NUMBER (PK); FLIGHT_ID NUMBER; DELAY_MIN NUMBER; REASON_CODE VARCHAR2; EVENT_DATE TIMESTAMP; EVENT_STATUS VARCHAR2	FK FLIGHT_ID; nonnegative delay.

The application creates APP_LOGIN_USERS separately when needed. Its identity key and password hash support local authentication; it is an application-support table rather than a tenth airline-domain entity.

Database platform used by the application

Review 0 records MySQL 8.0 in the initial software-requirement table. The delivered server.py and README use Oracle XE through python-oracledb Thin mode. This Review 1 implementation therefore documents Oracle XE as the actual runtime. SQL dialect, identity generation and constraints must match the Oracle schema used by the application.

FRONT END AND BACK END CONNECTIVITY PROCEDURE

The application runs locally. A browser loads index.html from FastAPI; JavaScript sends JSON requests to the API; the service maps allowed presentation fields to Oracle columns and uses python-oracledb connections to retrieve or write records.

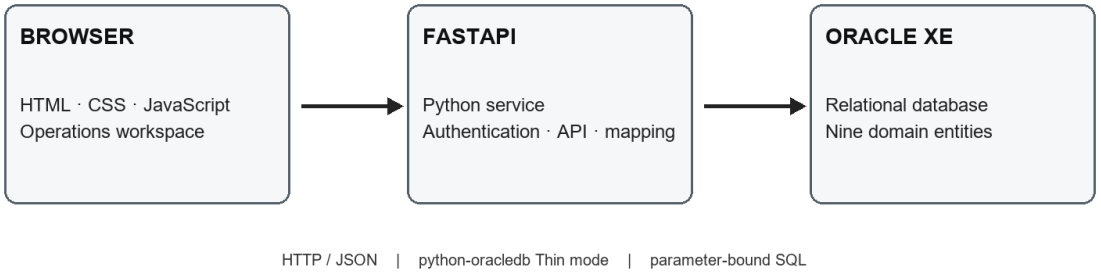


Figure 2. Overall system design and database connectivity.

Connectivity sequence

Step	Procedure
1. Configure	Set Oracle username, password, service DSN and session secret in private environment settings.
2. Start	Run the FastAPI application locally at 127.0.0.1:8001.
3. Authenticate	Register or sign in; the service checks a password hash and issues a signed HttpOnly cookie.
4. Read	The browser requests current operational data; the API queries Oracle using a connection pool.
5. Write	Create/update/delete requests use mapped fields and bound values; successful writes are committed.
6. Refresh	The browser reloads Oracle data; the project README describes automatic refresh every five seconds.

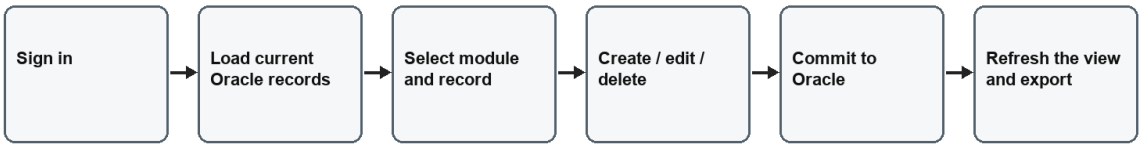


Figure 3. Authenticated operational data workflow.

Authentication and security notes

Passwords use scrypt hashes and signed HttpOnly cookies expire after 12 hours. All accounts share the same permissions; roles are not implemented. The app is for local coursework and needs production security before network exposure.

PRACTICAL IMPLEMENTATION, BUSINESS AND SUSTAINABILITY

Practical implementation

The stack comprises a browser interface (HTML, CSS and JavaScript), a Python FastAPI service, python-oracledb in Thin mode and Oracle XE. The API provides health and authentication endpoints, consolidated data reads, reference lookups and entity-level create, update and delete routes. Parameter binding is used for values, and database constraints remain authoritative for relationships and valid domains.

The supplied evidence shows a flight create flow from the Add Flight form through Oracle insertion, then a flight update reflected in a SQL query. Delete is represented by the browser confirmation prompt; the screenshot set does not show the subsequent deletion result. Key generation depends on the deployed identity/default/trigger configuration and should be confirmed against Oracle metadata.

Business aspect

Potential users include flight operations coordinators, fleet planners, cargo teams and sustainability analysts. A shared view can reduce manual switching among separate lists and make linked flight, fleet, cargo and emissions records easier to review. These are potential benefits; the coursework demonstration has not measured time savings or operational outcomes.

Business path	Application	Requirements before adoption
Internal operations tool	Deploy within an airline operations team.	Integration, access control, audit, uptime and governance.
Subscription service	Offer modules by airline size or user count.	Tenant isolation, cloud security, support and compliance.
Modular services	Add cargo monitoring, disruption analytics or reporting.	Verified data definitions and reliable source integrations.
Implementation support	Provide setup, data migration and training.	Domain validation and ongoing maintenance.

Cost drivers include database and service hosting, security review, integration work, identity management, monitoring, backup, support and training. A pilot should measure task time, data error rates,

disruption response time and the completeness of fuel and emissions records before making financial claims.

Sustainability inspiration and practical scope

The project was inspired by making sustainability information part of day-to-day operational records. Fuel and CO2 logging can give analysts a starting point for comparing flights, routes and aircraft. Required and current cargo temperatures can support visibility for medical and perishable shipments. Better information can guide decisions, but the application does not itself reduce emissions, optimize routes or prove avoided spoilage.

Future sustainability work should define verified emission factors, data provenance, reporting boundaries, route distance and passenger/cargo load. Potential indicators include fuel per flight-distance, CO2 per passenger-kilometre or tonne-kilometre, cold-chain excursion rate and completeness of emission logs. External sustainability claims should be based on traceable and validated data.

FULL IMPLEMENTATION CODE

The following appendices reproduce the supplied implementation source. Private .env credentials, session secrets and local probes are excluded.

APPENDIX A SERVER SIDE IMPLEMENTATION

```
0001  import os
0002  import base64
0003  import hashlib
0004  import hmac
0005  import json
0006  import re
0007  import secrets
0008  from contextlib import contextmanager
0009  from datetime import date, datetime
0010  from pathlib import Path
0011  from typing import Any
0012
0013  import oracledb
0014  from dotenv import load_dotenv
0015  from fastapi import Depends, FastAPI, HTTPException, Request, Response
0016  from fastapi.responses import FileResponse
0017
0018  ROOT = Path(__file__).resolve().parent
0019  load_dotenv(ROOT / ".env")
0020
0021  app = FastAPI(title="Northstar Oracle API")
0022  _pool: oracledb.ConnectionPool | None = None
0023  AUTH_TABLE = "APP_LOGIN_USERS"
0024  SESSION_COOKIE = "northstar_session"
0025  SESSION_SECONDS = 12 * 60 * 60
0026  _session_secret: bytes | None = None
0027
0028  ENTITIES: dict[str, dict[str, Any]] = {
0029      "flights": {
0030          "table": "FLIGHT", "key": "FLIGHT_ID",
0031          "fields": {"route": "ROUTE_CODE", "departure": "DEP_TIME", "arrival": "ARR_TIME",
"aircraft_id": "AIRCRAFT_ID", "status": "FLIGHT_STATUS"},
0032      },
0033      "fleet": {
0034          "table": "AIRCRAFT", "key": "AIRCRAFT_ID",
0035          "fields": {"model": "MODEL", "capacity_kg": "CAPACITY_KG", "efficiency":
"FUEL_EFFICIENCY", "status": "AIRCRAFT_STATUS"},
0036      },
0037      "crew": {
0038          "table": "CREW", "key": "CREW_ID",
0039          "fields": {"name": "CREW_NAME", "role": "ROLE", "base": "BASE_LOCATION"},
0040      },
0041      "cargo": {
0042          "table": "SPEC_CARGO", "key": "CARGO_ID",
0043          "fields": {"flight_id": "FLIGHT_ID", "name": "CARGO_DESCRIPTION", "weight_kg":
"WEIGHT_KG", "category": "CATEGORY", "required_temp_c": "REQ_TEMP_C", "current_temp_c":
"CURRENT_TEMP_C", "tracking": "TRACKING_STATUS"},
0044      },
0045      "passengers": {
0046          "table": "PASSENGER", "key": "PASSENGER_ID",
0047          "fields": {"name": "PASSENGER_NAME", "passport_no": "PASSPORT_NO", "tier_id":
"TIER_ID", "baggage_kg": "BAGGAGE_KG"},
0048      },
0049      "disruptions": {
0050          "table": "DISRUPTION", "key": "EVENT_ID",
0051          "fields": {"flight_id": "FLIGHT_ID", "delay_min": "DELAY_MIN", "reason":
"REASON_CODE", "event_date": "EVENT_DATE", "status": "EVENT_STATUS"},
0052      },
0053      "emissions": {
0054          "table": "EMISSIONS", "key": "LOG_ID",
0055          "fields": {"flight_id": "FLIGHT_ID", "fuel_liters": "FUEL_LITERS", "co2_tons":
"CO2_TONS", "logged_at": "LOGGED_AT"},
0056      },
0057  }
0058
0059
0060  def get_pool() -> oracledb.ConnectionPool:
0061      global _pool
0062      if _pool is None:
0063          user = os.getenv("ORACLE_USER")
0064          password = os.getenv("ORACLE_PASSWORD")
0065          dsn = os.getenv("ORACLE_DSN")
```

```
0066          if not user or not password or not dsn:
0067              raise HTTPException(status_code=503, detail="Set ORACLE_USER, ORACLE_PASSWORD, and
ORACLE_DSN in the local .env file.")
0068          try:
0069              _pool = oracledb.create_pool(user=user, password=password, dsn=dsn, min=1, max=4,
increment=1)
0070          except oracledb.Error as exc:
0071              raise HTTPException(status_code=503, detail=f"Oracle connection failed: {exc}")
from exc
0072      return _pool
0073
0074
0075  @contextmanager
0076  def connection():
0077      try:
0078          conn = get_pool().acquire()
0079      except oracledb.Error as exc:
0080          raise HTTPException(status_code=503, detail=f"Cannot acquire an Oracle connection:
{exc}") from exc
0081      try:
0082          yield conn
0083      finally:
0084          conn.close()
0085
0086
0087  def rows_as_dicts(cursor: oracledb.Cursor) -> list[dict[str, Any]]:
0088      names = [column[0].lower() for column in cursor.description]
0089      result = []
0090      for row in cursor:
0091          item = dict(zip(names, row))
0092          for key, value in item.items():
0093              if isinstance(value, (datetime, date)):
0094                  item[key] = value.isoformat()
0095          result.append(item)
0096      return result
0097
0098
0099  def ensure_auth_table() -> None:
0100      with connection() as conn:
0101          with conn.cursor() as cursor:
0102              cursor.execute("SELECT COUNT(*) FROM USER_TABLES WHERE TABLE_NAME = :table_name",
{"table_name": AUTH_TABLE})
0103              if cursor.fetchone()[0]:
0104                  return
0105              try:
0106                  cursor.execute(f"""CREATE TABLE {AUTH_TABLE} (
0107                      USER_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
0108                      NAME VARCHAR2(80) NOT NULL,
0109                      EMAIL VARCHAR2(254) NOT NULL UNIQUE,
0110                      PASSWORD_HASH VARCHAR2(256) NOT NULL,
0111                      CREATED_AT TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
0112                  )""")
0113              except oracledb.DatabaseError as exc:
0114                  if getattr(exc.args[0], "code", None) != 955:
0115                      raise
0116
0117
0118  def hash_password(password: str, salt: bytes | None = None) -> str:
0119      salt = salt or secrets.token_bytes(16)
0120      digest = hashlib.scrypt(password.encode("utf-8"), salt=salt, n=2**14, r=8, p=1, dklen=64)
0121      return
f"scrypt$16384$8$1${base64.urlsafe_b64encode(salt).decode()}${base64.urlsafe_b64encode(digest).decod
e()}"
0122
0123
0124  def verify_password(password: str, encoded: str) -> bool:
0125      try:
0126          algorithm, n, r, p, salt_text, digest_text = encoded.split("$", 5)
0127          if algorithm != "scrypt":
0128              return False
0129          salt = base64.urlsafe_b64decode(salt_text.encode())
0130          expected = base64.urlsafe_b64decode(digest_text.encode())
0131          actual = hashlib.scrypt(password.encode("utf-8"), salt=salt, n=int(n), r=int(r),
p=int(p), dklen=len(expected))
0132          return hmac.compare_digest(actual, expected)
0133      except (ValueError, TypeError):
0134          return False
0135
0136
0137  def get_session_secret() -> bytes:
0138      global _session_secret
0139      if _session_secret is None:
0140          path = ROOT / ".session_secret"
0141          try:
0142              _session_secret = path.read_bytes()
0143          except FileNotFoundError:
0144              _session_secret = secrets.token_bytes(32)
```

```
0145              try:
0146                  with path.open("xb") as secret_file:
0147                      secret_file.write(_session_secret)
0148              except FileExistsError:
0149                  _session_secret = path.read_bytes()
0150      return _session_secret
0151
0152
0153  def create_session(user: dict[str, Any]) -> str:
0154      payload = {"id": user["user_id"], "name": user["name"], "email": user["email"], "exp":
int(datetime.now().timestamp()) + SESSION_SECONDS}
0155      encoded = base64.urlsafe_b64encode(json.dumps(payload, separators=(",",
":")).encode()).rstrip(b"=")
0156      signature = hmac.new(get_session_secret(), encoded, hashlib.sha256).digest()
0157      return f"{encoded.decode()}.{base64.urlsafe_b64encode(signature).rstrip(b'=').decode()}"
0158
0159
0160  def session_user(request: Request) -> dict[str, Any] | None:
0161      token = request.cookies.get(SESSION_COOKIE)
0162      if not token or "." not in token:
0163          return None
0164      encoded_text, signature_text = token.rsplit(".", 1)
0165      encoded = encoded_text.encode()
0166      expected = hmac.new(get_session_secret(), encoded, hashlib.sha256).digest()
0167      try:
0168          supplied = base64.urlsafe_b64decode(signature_text + "=" * (-len(signature_text) % 4))
0169          payload_bytes = base64.urlsafe_b64decode(encoded_text + "=" * (-len(encoded_text) %
4))
0170          payload = json.loads(payload_bytes)
0171      except (ValueError, json.JSONDecodeError):
0172          return None
0173      if not hmac.compare_digest(expected, supplied) or payload.get("exp", 0) <
datetime.now().timestamp():
0174          return None
0175      return payload
0176
0177
0178  def require_user(request: Request) -> dict[str, Any]:
0179      user = session_user(request)
0180      if user is None:
0181          raise HTTPException(status_code=401, detail="Please sign in to continue.")
0182      return user
0183
0184
0185  def flight_label(flight_id: int | None) -> str:
0186      return f"NS {flight_id:03d}" if flight_id is not None else "—"
0187
0188
0189  def read_all() -> dict[str, list[dict[str, Any]]]:
0190      queries = {
0191          "flights": """SELECT f.FLIGHT_ID AS db_id, f.ROUTE_CODE AS route,
0192              TO_CHAR(f.DEP_TIME, 'YYYY-MM-DD\"T\"HH24:MI:SS') AS departure_iso,
0193              TO_CHAR(f.DEP_TIME, 'HH24:MI') AS departure,
0194              TO_CHAR(f.ARR_TIME, 'YYYY-MM-DD\"T\"HH24:MI:SS') AS arrival_iso,
0195              TO_CHAR(f.ARR_TIME, 'HH24:MI') AS arrival,
0196              f.AIRCRAFT_ID AS aircraft_id, a.MODEL AS model, f.FLIGHT_STATUS AS status
0197              FROM FLIGHT f JOIN AIRCRAFT a ON a.AIRCRAFT_ID = f.AIRCRAFT_ID ORDER BY
f.DEP_TIME""",
0198          "fleet": """SELECT a.AIRCRAFT_ID AS db_id, a.MODEL AS model, a.CAPACITY_KG AS
capacity_kg,
0199              a.FUEL_EFFICIENCY AS efficiency, a.AIRCRAFT_STATUS AS status,
0200              (SELECT MIN(f.FLIGHT_ID) FROM FLIGHT f WHERE f.AIRCRAFT_ID = a.AIRCRAFT_ID) AS
flight_id
0201              FROM AIRCRAFT a ORDER BY a.AIRCRAFT_ID""",
0202          "crew": """SELECT c.CREW_ID AS db_id, c.CREW_NAME AS name, c.ROLE AS role,
0203              c.BASE_LOCATION AS base, fc.FLIGHT_ID AS flight_id, fc.SHIFT_HOURS AS shift
0204              FROM CREW c LEFT JOIN (SELECT CREW_ID, MIN(FLIGHT_ID) AS FLIGHT_ID,
0205              MIN(SHIFT_HOURS) AS SHIFT_HOURS FROM FLIGHT_CREW GROUP BY CREW_ID) fc
0206              ON fc.CREW_ID = c.CREW_ID ORDER BY c.CREW_ID""",
0207          "cargo": """SELECT c.CARGO_ID AS db_id, c.CARGO_DESCRIPTION AS name, c.CATEGORY AS
category,
0208              c.FLIGHT_ID AS flight_id, c.WEIGHT_KG AS weight_kg, c.REQ_TEMP_C AS
required_temp_c,
0209              c.CURRENT_TEMP_C AS current_temp_c, c.TRACKING_STATUS AS tracking
0210              FROM SPEC_CARGO c ORDER BY c.CARGO_ID""",
0211          "passengers": """SELECT p.PASSENGER_ID AS db_id, p.PASSENGER_NAME AS name,
0212              p.TIER_ID AS tier_id, lt.TIER_NAME AS tier, lt.EXTRA_BAGGAGE_KG AS allowance_kg,
0213              p.BAGGAGE_KG AS baggage_kg,
0214              (SELECT MIN(b.FLIGHTNO) FROM BOOKING b WHERE b.P_ID = TO_CHAR(p.PASSENGER_ID)) AS
flight_id
0215              FROM PASSENGER p JOIN LOYALTY_TIER lt ON lt.TIER_ID = p.TIER_ID ORDER BY
p.PASSENGER_ID""",
0216          "disruptions": """SELECT d.EVENT_ID AS db_id, d.FLIGHT_ID AS flight_id, f.ROUTE_CODE
AS route,
0217              d.REASON_CODE AS reason, d.DELAY_MIN AS delay_min,
0218              TO_CHAR(d.EVENT_DATE, 'YYYY-MM-DD\"T\"HH24:MI:SS') AS event_date,
```

```
0219              d.EVENT_STATUS AS status FROM DISRUPTION d JOIN FLIGHT f ON f.FLIGHT_ID =
d.FLIGHT_ID
0220              ORDER BY d.EVENT_DATE DESC""",
0221          "emissions": """SELECT e.LOG_ID AS db_id, e.FLIGHT_ID AS flight_id, f.ROUTE_CODE AS
route,
0222              e.FUEL_LITERS AS fuel_liters, e.CO2_TONS AS co2_tons, f.AIRCRAFT_ID AS
aircraft_id,
0223              TO_CHAR(e.LOGGED_AT, 'YYYY-MM-DD\"T\"HH24:MI:SS') AS logged_at
0224              FROM EMISSIONS e JOIN FLIGHT f ON f.FLIGHT_ID = e.FLIGHT_ID ORDER BY e.LOGGED_AT
DESC""",
0225          "loyalty_tiers": "SELECT TIER_ID AS tier_id, TIER_NAME AS tier_name, EXTRA_BAGGAGE_KG
AS allowance_kg FROM LOYALTY_TIER ORDER BY TIER_ID",
0226      }
0227      result: dict[str, list[dict[str, Any]]] = {}
0228      with connection() as conn:
0229          with conn.cursor() as cursor:
0230              for entity, sql in queries.items():
0231                  cursor.execute(sql)
0232                  records = rows_as_dicts(cursor)
0233                  if entity == "flights":
0234                      for row in records:
0235                          row.update(id=flight_label(row["db_id"]), aircraft=f"AC-
{row['aircraft_id']:03d} · {row['model']}")
0236                  elif entity == "fleet":
0237                      for row in records:
0238                          row.update(id=f"AC-{row['db_id']:03d}",
capacity=f"{row['capacity_kg']:,.0f} kg", efficiency=f"{row['efficiency']} L / 100 km",
flights=flight_label(row["flight_id"]) if row["flight_id"] else "—")
0239                  elif entity == "crew":
0240                      for row in records:
0241                          row.update(id=f"CR-{row['db_id']:03d}",
flight=flight_label(row["flight_id"]) if row["flight_id"] else "—", shift=f"{row['shift']} h" if
row["shift"] is not None else "—", status="On duty" if row["flight_id"] else "Standby")
0242                  elif entity == "cargo":
0243                      for row in records:
0244                          row.update(id=f"CG-{row['db_id']:04d}",
flight=flight_label(row["flight_id"]), weight=f"{row['weight_kg']:,.0f} kg",
required=f"{row['required_temp_c']} °C" if row["required_temp_c"] is not None else "Not specified",
current=f"{row['current_temp_c']} °C" if row["current_temp_c"] is not None else "Not recorded")
0245                  elif entity == "passengers":
0246                      for row in records:
0247                          row.update(id=f"P-{row['db_id']:05d}",
allowance=f"+{row['allowance_kg']:g} kg", baggage=f"{row['baggage_kg']:g} kg",
flight=flight_label(row["flight_id"]) if row["flight_id"] else "—")
0248                  elif entity == "disruptions":
0249                      for row in records:
0250                          row.update(id=f"DP-{row['db_id']:04d}",
flight=flight_label(row["flight_id"]), delay=f"{row['delay_min']} min", date=row["event_date"])
0251                  elif entity == "emissions":
0252                      for row in records:
0253                          row.update(id=f"EM-{row['db_id']:04d}",
flight=flight_label(row["flight_id"]), fuel=f"{row['fuel_liters']:,.0f} L",
co2=f"{row['co2_tons']:g} t", aircraft=f"AC-{row['aircraft_id']:03d}", logged=row["logged_at"])
0254                  result[entity] = records
0255      return result
0256
0257
0258  @app.get("/api/health")
0259  def health() -> dict[str, Any]:
0260      with connection() as conn:
0261          with conn.cursor() as cursor:
0262              cursor.execute("SELECT USER, SYS_CONTEXT('USERENV','SERVICE_NAME'),
SYS_CONTEXT('USERENV','CON_NAME') FROM DUAL")
0263              user, service, container = cursor.fetchone()
0264              cursor.execute("SELECT TABLE_NAME FROM USER_TABLES")
0265              visible = {row[0] for row in cursor.fetchall()}
0266      required = {"AIRCRAFT", "CREW", "DISRUPTION", "EMISSIONS", "FLIGHT", "LOYALTY_TIER",
"PASSENGER", "SPEC_CARGO"}
0267      missing = sorted(required - visible)
0268      if missing:
0269          raise HTTPException(status_code=503, detail=f"Oracle is reachable as {user} on
{service}/{container}, but this schema cannot see the airline tables. Use the same Oracle username
and service as the SQL*Plus session where you ran the inserts.")
0270      return {"status": "ok", "database": "Oracle", "user": user, "service": service,
"container": container}
0271
0272
0273  @app.post("/api/auth/signup", status_code=201)
0274  def signup(payload: dict[str, Any], response: Response) -> dict[str, Any]:
0275      name = str(payload.get("name", "")).strip()
0276      email = str(payload.get("email", "")).strip().lower()
0277      password = str(payload.get("password", ""))
0278      if not name or len(name) > 80:
0279          raise HTTPException(status_code=422, detail="Name is required and must be 80
characters or fewer.")
0280      if not re.fullmatch(r"[^\s@]+@[^\s@]+\.[^\s@]+", email) or len(email) > 254:
0281          raise HTTPException(status_code=422, detail="Enter a valid email address.")
```

```
0282      if len(password) < 10:
0283          raise HTTPException(status_code=422, detail="Password must be at least 10
characters.")
0284      ensure_auth_table()
0285      try:
0286          with connection() as conn:
0287              with conn.cursor() as cursor:
0288                  user_id_var = cursor.var(oracledb.NUMBER)
0289                  cursor.execute(
0290                      f"INSERT INTO {AUTH_TABLE} (NAME, EMAIL, PASSWORD_HASH) VALUES (:name,
:email, :password_hash) RETURNING USER_ID INTO :user_id",
0291                      {"name": name, "email": email, "password_hash": hash_password(password),
"user_id": user_id_var},
0292                  )
0293                  user_id = user_id_var.getvalue()
0294                  conn.commit()
0295      except oracledb.DatabaseError as exc:
0296          if getattr(exc.args[0], "code", None) == 1:
0297              raise HTTPException(status_code=409, detail="An account with this email already
exists.") from exc
0298          raise HTTPException(status_code=503, detail="Could not create the account in Oracle.")
from exc
0299      if isinstance(user_id, list):
0300          user_id = user_id[0]
0301      user = {"user_id": int(user_id), "name": name, "email": email}
0302      response.set_cookie(SESSION_COOKIE, create_session(user), httponly=True, samesite="lax",
secure=False, max_age=SESSION_SECONDS, path="/")
0303      return user
0304
0305
0306  @app.post("/api/auth/login")
0307  def login(payload: dict[str, Any], response: Response) -> dict[str, Any]:
0308      email = str(payload.get("email", "")).strip().lower()
0309      password = str(payload.get("password", ""))
0310      ensure_auth_table()
0311      try:
0312          with connection() as conn:
0313              with conn.cursor() as cursor:
0314                  cursor.execute(f"SELECT USER_ID, NAME, EMAIL, PASSWORD_HASH FROM {AUTH_TABLE}
WHERE EMAIL = :email", {"email": email})
0315                  row = cursor.fetchone()
0316      except oracledb.DatabaseError as exc:
0317          raise HTTPException(status_code=503, detail="Could not check the account in Oracle.")
from exc
0318      if row is None or not verify_password(password, row[3]):
0319          raise HTTPException(status_code=401, detail="Email or password is incorrect.")
0320      user = {"user_id": int(row[0]), "name": row[1], "email": row[2]}
0321      response.set_cookie(SESSION_COOKIE, create_session(user), httponly=True, samesite="lax",
secure=False, max_age=SESSION_SECONDS, path="/")
0322      return user
0323
0324
0325  @app.get("/api/auth/me")
0326  def current_user(user: dict[str, Any] | None = Depends(session_user)) -> dict[str, Any]:
0327      if user is None:
0328          raise HTTPException(status_code=401, detail="Please sign in to continue.")
0329      return user
0330
0331
0332  @app.post("/api/auth/logout")
0333  def logout(response: Response) -> dict[str, str]:
0334      response.delete_cookie(SESSION_COOKIE, path="/")
0335      return {"message": "Signed out"}
0336
0337
0338  @app.get("/api/data")
0339  def get_data(_user: dict[str, Any] = Depends(require_user)) -> dict[str, list[dict[str,
Any]]]:
0340      try:
0341          return read_all()
0342      except oracledb.DatabaseError as exc:
0343          error = exc.args[0]
0344          if getattr(error, "code", None) == 942:
0345              raise HTTPException(status_code=503, detail="Oracle is reachable, but the
configured schema cannot see one or more airline tables. Use the same Oracle username and service as
the SQL*Plus session where you ran the inserts.") from exc
0346          raise HTTPException(status_code=503, detail=f"Oracle query failed: {error}") from exc
0347
0348
0349  @app.get("/api/reference/{entity}")
0350  def get_reference(entity: str, _user: dict[str, Any] = Depends(require_user)) ->
list[dict[str, Any]]:
0351      queries = {
0352          "flights": "SELECT FLIGHT_ID AS db_id, ROUTE_CODE AS route FROM FLIGHT ORDER BY
FLIGHT_ID",
0353          "fleet": "SELECT AIRCRAFT_ID AS db_id, MODEL AS model FROM AIRCRAFT ORDER BY
AIRCRAFT_ID",
```

```
0354          "tiers": "SELECT TIER_ID AS tier_id, TIER_NAME AS tier_name FROM LOYALTY_TIER ORDER BY
TIER_ID",
0355      }
0356      sql = queries.get(entity)
0357      if sql is None:
0358          raise HTTPException(status_code=404, detail="Unknown reference list.")
0359      with connection() as conn:
0360          with conn.cursor() as cursor:
0361              cursor.execute(sql)
0362              return rows_as_dicts(cursor)
0363
0364
0365  def get_entity(entity: str) -> dict[str, Any]:
0366      definition = ENTITIES.get(entity)
0367      if definition is None:
0368          raise HTTPException(status_code=404, detail="Unknown entity.")
0369      return definition
0370
0371
0372  def write_record(entity: str, key_value: int | str | None, payload: dict[str, Any], create:
bool) -> dict[str, Any]:
0373      definition = get_entity(entity)
0374      fields: dict[str, str] = definition["fields"]
0375      provided_key = payload.pop("db_id", None) if create else None
0376      unknown = set(payload) - set(fields)
0377      if unknown:
0378          raise HTTPException(status_code=422, detail=f"Unsupported fields: {',
'.join(sorted(unknown))}")
0379      if provided_key is not None:
0380          try:
0381              provided_key = int(provided_key)
0382          except (TypeError, ValueError) as exc:
0383              raise HTTPException(status_code=422, detail="db_id must be a number.") from exc
0384      if not payload:
0385          raise HTTPException(status_code=422, detail="Provide at least one field.")
0386      columns = list(payload)
0387      binds = {name: payload[name] for name in columns}
0388      date_fields = {"departure", "arrival", "event_date", "logged_at"}
0389      for name in columns:
0390          if name in date_fields and binds[name] is not None:
0391              try:
0392                  binds[name] = datetime.fromisoformat(str(binds[name]))
0393              except ValueError as exc:
0394                  raise HTTPException(status_code=422, detail=f"{name} must be an ISO
date/time.") from exc
0395      try:
0396          with connection() as conn:
0397              with conn.cursor() as cursor:
0398                  if create:
0399                      column_sql = ", ".join(fields[name] for name in columns)
0400                      bind_sql = ", ".join(f":{name}" for name in columns)
0401                      if provided_key is not None:
0402                          column_sql = f"{definition['key']}, {column_sql}"
0403                          bind_sql = f":provided_key, {bind_sql}"
0404                          binds["provided_key"] = provided_key
0405                      key_var = cursor.var(oracledb.NUMBER)
0406                      sql = f"INSERT INTO {definition['table']} ({column_sql}) VALUES
({bind_sql}) RETURNING {definition['key']} INTO :new_key"
0407                      cursor.execute(sql, {**binds, "new_key": key_var})
0408                      new_key = key_var.getvalue()
0409                      conn.commit()
0410                      return {"db_id": new_key, "message": "Created"}
0411                  assignments = ", ".join(f"{fields[name]} = :{name}" for name in columns)
0412                  binds["record_key"] = key_value
0413                  cursor.execute(f"UPDATE {definition['table']} SET {assignments} WHERE
{definition['key']} = :record_key", binds)
0414                  if cursor.rowcount == 0:
0415                      raise HTTPException(status_code=404, detail="Record not found.")
0416                  conn.commit()
0417                  return {"db_id": key_value, "message": "Updated"}
0418      except HTTPException:
0419          raise
0420      except oracledb.Error as exc:
0421          raise HTTPException(status_code=409, detail=f"Oracle rejected the change. Check
required fields, relationships, and generated IDs: {exc}") from exc
0422
0423
0424  @app.post("/api/{entity}", status_code=201)
0425  def create_record(entity: str, payload: dict[str, Any], _user: dict[str, Any] =
Depends(require_user)) -> dict[str, Any]:
0426      return write_record(entity, None, payload, True)
0427
0428
0429  @app.put("/api/{entity}/{record_id}")
0430  def update_record(entity: str, record_id: str, payload: dict[str, Any], _user: dict[str, Any]
= Depends(require_user)) -> dict[str, Any]:
0431      definition = get_entity(entity)
```

```
0432      try:
0433          key_value: int | str = int(record_id) if definition["key"] != "BOOKING_ID" else
record_id
0434      except ValueError as exc:
0435          raise HTTPException(status_code=422, detail="Record ID must be numeric.") from exc
0436      return write_record(entity, key_value, payload, False)
0437
0438
0439  @app.delete("/api/{entity}/{record_id}")
0440  def delete_record(entity: str, record_id: str, _user: dict[str, Any] = Depends(require_user))
-> dict[str, str]:
0441      definition = get_entity(entity)
0442      try:
0443          key_value = int(record_id)
0444      except ValueError as exc:
0445          raise HTTPException(status_code=422, detail="Record ID must be numeric.") from exc
0446      try:
0447          with connection() as conn:
0448              with conn.cursor() as cursor:
0449                  cursor.execute(f"DELETE FROM {definition['table']} WHERE {definition['key']} =
:record_key", {"record_key": key_value})
0450                  if cursor.rowcount == 0:
0451                      raise HTTPException(status_code=404, detail="Record not found.")
0452                  conn.commit()
0453          return {"message": "Deleted"}
0454      except HTTPException:
0455          raise
0456      except oracledb.Error as exc:
0457          raise HTTPException(status_code=409, detail=f"Oracle rejected the delete, possibly
because related records exist: {exc}") from exc
0458
0459
0460  @app.get("/")
0461  def index() -> FileResponse:
0462      return FileResponse(ROOT / "index.html")
0463
0464
0465  @app.get("/{asset_path:path}")
0466  def static_files(asset_path: str) -> FileResponse:
0467      path = (ROOT / asset_path).resolve()
0468      if ROOT not in path.parents or not path.is_file():
0469          raise HTTPException(status_code=404, detail="Not found.")
0470      return FileResponse(path)
```

APPENDIX B BROWSER SIDE IMPLEMENTATION

```
0001  <!doctype html>
0002  <html lang="en">
0003  <head>
0004  <meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-
scale=1"><meta name="theme-color" content="#0b1421">
0005  <title>Northstar · Airline Logistics</title>
0006  <style>
0007  :root{--nav:#0b1421;--nav2:#121f30;--ink:#172638;--muted:#7f8b98;--line:#e7edf0;--bg:#f4f7f8;-
-teal:#17b394;--teal2:#e7f7f3;--amber:#d99438;--red:#d36c62;--shadow:0 18px 45px #1726380b}*{box-
sizing:border-box}body{margin:0;background:var(--bg);font:14px/1.45 "Segoe UI",Arial,sans-
serif;color:var(--
ink)}button,input,select{font:inherit}button{cursor:pointer}.layout{display:grid;grid-template-
columns:252px 1fr;min-height:100vh}.sidebar{background:var(--nav);color:#dbe4ed;padding:22px 15px
16px;display:flex;flex-direction:column;min-
height:100vh;position:sticky;top:0;height:100vh;overflow:auto}.brand{display:flex;align-
items:center;gap:11px;padding:2px 12px 28px;color:white;font-weight:800;letter-spacing:.13em;font-
size:14px}.brand-mark{font-size:25px;color:#37d3b6}.brand small{display:block;color:#8293a4;font-
size:9px;letter-spacing:.18em;margin-top:3px;font-weight:600}.workspace-
label,.caption{color:#78899b;font-weight:700;letter-spacing:.17em;font-size:9px;padding:0 12px
9px}.workspace{display:flex;gap:10px;align-items:center;padding:10px;background:#142438;border:1px
solid #26364a;border-radius:9px;margin-bottom:23px;font-size:11px;font-weight:700}.workspace-
badge{display:grid;place-
items:center;width:32px;height:32px;background:#234154;color:#69d8c0;border-radius:8px;font-
size:10px}.workspace small,.profile small{display:block;color:#8da0b1;font-weight:400;margin-
top:3px}.caption{margin:9px 0 3px}.nav{display:grid;gap:3px}.nav
button{border:0;background:transparent;color:#a3b0be;display:flex;align-items:center;gap:11px;text-
align:left;border-radius:7px;padding:10px 12px;font-size:12px}.nav
button:hover{background:#142337;color:white}.nav
button.active{background:#17382f;color:#6be0c4;font-weight:700}.nav .ico{width:17px;text-
align:center;font-size:16px}.nav .count{margin-left:auto;padding:2px 7px;border-
radius:20px;background:#25374a;font-size:9px}.nav
.count.alert{background:#432c34;color:#f29c91}.side-bottom{margin-
top:auto}.brief{padding:14px;margin:12px 4px 14px;border-radius:9px;background:linear-
gradient(145deg,#173b3a,#152c30);border:1px solid #294a48}.brief .spark{font-
size:18px;color:#64d9bc}.brief strong{display:block;font-size:12px;margin:8px 0 4px}.brief p{font-
size:10px;color:#adc0bf;margin:0;line-height:1.55}.profile{display:flex;align-
items:center;gap:9px;padding:13px 7px 0;border-top:1px solid #263547;font-size:11px;font-
weight:700}.avatar{display:grid;place-
items:center;width:31px;height:31px;background:#dcece9;color:#247d70;border-radius:50%;font-
size:10px}.profile small{font-size:10px}.main{min-
width:0}.topbar{height:70px;background:#fff;border-bottom:1px solid var(--line);display:flex;align-
items:center;justify-content:space-between;padding:0 34px;gap:16px}.crumb{font-
size:12px;color:#82909d}.crumb b{color:#18283a;margin-left:10px}.top-actions{display:flex;align-
items:center;gap:15px}.live{color:#258d78;font-size:10px;display:flex;align-
items:center;gap:7px;white-space:nowrap}.live i{width:7px;height:7px;background:#29bf99;border-
radius:50%;box-shadow:0 0 0 4px #e4f6f0}.btn{border:1px solid #e1e8eb;background:#fff;border-
radius:6px;padding:8px 11px;color:#526273;font-size:11px}.btn:hover{border-
color:#98cfc2;color:#158a74}.btn.primary{background:#159d83;border-color:#159d83;color:white;font-
weight:700}.top-avatar{display:grid;place-items:center;width:32px;height:32px;border-
radius:50%;background:#dcece9;color:#247d70;font-weight:700;font-size:10px}.content{padding:30px
34px 42px;max-width:1550px;margin:auto}.eyebrow{font-size:9px;letter-spacing:.17em;font-
weight:800;color:#778897}.title-row{display:flex;justify-content:space-between;align-items:flex-
end;gap:16px;margin-bottom:22px}.title-row h1{font-size:25px;letter-spacing:-.04em;margin:5px 0
3px;color:#142437}.title-row p{font-size:11px;color:var(--
muted);margin:0}.actions{display:flex;gap:8px}.stats{display:grid;grid-template-
columns:repeat(4,minmax(0,1fr));gap:13px;margin-bottom:15px}.stat,.panel{background:#fff;border:1px
solid var(--line);border-radius:10px;box-shadow:var(--shadow)}.stat{padding:16px 17px;min-
height:112px}.stat-head{display:flex;justify-content:space-between;color:#607181;font-
size:10px;font-weight:600}.stat-icon{display:grid;place-items:center;width:27px;height:27px;border-
radius:7px;background:#e8f6f2;color:#168c77;font-size:14px}.stat-
icon.blue{background:#eef2fc;color:#6079c1}.stat-icon.amber{background:#fbf2e4;color:#c18b3a}.stat-
icon.red{background:#fbebea;color:#c96f67}.stat-value{font-size:26px;font-weight:750;letter-
spacing:-.05em;margin-top:8px}.stat-foot{font-size:9px;color:#91a0aa;margin-top:1px}.overview-
grid{display:grid;grid-template-columns:minmax(0,1.55fr) minmax(280px,.9fr);gap:14px;margin-
bottom:14px}.panel{padding:17px 18px}.panel-head{display:flex;align-items:flex-start;justify-
content:space-between;gap:12px;margin-bottom:13px}.panel-title{font-size:12px;font-
weight:750;color:#26384a}.panel-sub{font-size:9px;color:#929da6;margin-top:3px}.text-
link{border:0;background:transparent;color:#159b83;font-size:10px;font-weight:700}.chart-
wrap{position:relative;width:100%;height:214px}.chart{display:block;width:100%;height:100%;overflow:
visible}.chart .gridline{stroke:#ebf0f1;stroke-width:1}.chart text{font:9px "Segoe UI",Arial,sans-
serif;fill:#98a4ad}.chart .line{fill:none;stroke:#19b69a;stroke-width:3;stroke-linecap:round;stroke-
linejoin:round}.chart .area{fill:url(#area)}.chart .dot{fill:white;stroke:#16a98d;stroke-
width:2}.tooltip{position:absolute;z-index:3;display:none;pointer-
events:none;background:#132337;color:#fff;border-radius:7px;padding:8px 10px;min-width:135px;font-
size:9px;line-height:1.5;box-shadow:0 8px 24px #13233735;transform:translate(-50%,-
110%)}.tooltip.show{display:block}.tooltip b{display:block;font-size:11px;margin:2px 0}.status-
list{display:grid;gap:0}.status-row{display:flex;align-items:center;gap:11px;padding:12px 0;border-
bottom:1px solid #eef1f2}.status-row:last-child{border-bottom:0}.status-symbol{display:grid;place-
items:center;width:30px;height:30px;border-radius:8px;background:#f0f5f6;color:#387c78;font-
size:14px}.status-copy{flex:1}.status-copy strong{display:block;font-size:10px}.status-copy
small{font-size:9px;color:#919da6}.status-value{text-align:right;font-weight:750;font-
size:14px}.status-value small{display:block;font-size:8px;color:#9aa5ad;font-weight:400}.section-
panel{margin-bottom:14px}.table-wrap{overflow:auto}table{border-collapse:collapse;width:100%;min-
width:650px}th{text-align:left;color:#8b98a2;text-transform:uppercase;font-size:8px;letter-
```

```
spacing:.08em;font-weight:700;padding:10px 9px;border-bottom:1px solid #e9eef0;white-
space:nowrap}td{padding:11px 9px;border-bottom:1px solid #f0f3f4;font-size:10px;color:#506071;white-
space:nowrap}tbody tr{cursor:pointer}tbody tr:hover{background:#f7fbfa}td:first-child{font-
weight:700;color:#26394c}.badge{display:inline-flex;padding:4px 8px;border-radius:20px;font-
size:8px;font-
weight:700;background:#edf2f3;color:#647480}.badge.green{background:#e6f5ee;color:#23886d}.badge.amb
er{background:#fbf1e0;color:#b37c28}.badge.red{background:#fbeae8;color:#bd5d57}.badge.blue{backgrou
nd:#edf1fb;color:#5f74b3}.badge.gray{background:#eef0f2;color:#687681}.lower-grid{display:grid;grid-
template-columns:1fr 1fr;gap:14px}.mini-row{display:flex;align-items:center;gap:10px;padding:10px
0;border-top:1px solid #eef1f2}.mini-mark{display:grid;place-
items:center;width:27px;height:27px;border-radius:7px;background:#fff0eb;color:#cb7564}.mini-row
strong{display:block;font-size:10px}.mini-row small{display:block;color:#909da6;font-
size:9px;margin-top:3px}.mini-row .push{margin-left:auto;color:#b96d54;font-size:9px;font-
weight:700}.controls{display:flex;align-
items:center;gap:8px}.search{width:min(250px,42vw);border:1px solid #e4e9eb;background:#fff;border-
radius:6px;padding:8px 10px;font-size:10px;outline:none}.search:focus,select:focus{border-
color:#21aa90;box-shadow:0 0 0 3px #e6f6f2}select{border:1px solid #e4e9eb;background:#fff;border-
radius:6px;padding:8px 10px;color:#607181;font-size:10px}.empty{text-
align:center;color:#83909b;padding:42px 20px;font-size:11px}.modal-
bg{position:fixed;inset:0;display:none;align-items:center;justify-
content:center;padding:18px;background:#0b1421a8;z-index:20}.modal-
bg.open{display:flex}.modal{position:relative;background:#fff;border-
radius:11px;width:min(540px,100%);max-height:88vh;overflow:auto;padding:24px;box-shadow:0 28px 90px
#07111e55}.modal-close{position:absolute;right:14px;top:10px;border:0;background:transparent;font-
size:22px;color:#77838d}.modal h2{margin:6px 0;font-size:20px}.modal p{font-size:11px;color:var(--
muted);margin:0 0 16px}.detail-grid{display:grid;grid-template-columns:1fr
1fr;gap:1px;background:var(--line);border:1px solid var(--line);border-
radius:7px;overflow:hidden}.detail-cell{background:white;padding:11px}.detail-cell
small{display:block;color:#8a969f;font-size:8px;margin-bottom:5px}.detail-cell strong{font-
size:10px}.form-grid{display:grid;grid-template-columns:1fr 1fr;gap:12px}.form-grid label{font-
size:9px;color:#697986;font-weight:700}.form-grid input,.form-grid
select{display:block;width:100%;margin-top:5px;font-size:10px}.modal-actions{display:flex;justify-
content:flex-end;gap:8px;margin-
top:18px}.toast{position:fixed;right:22px;bottom:22px;background:#142438;color:white;padding:11px
15px;border-radius:7px;font-size:10px;box-shadow:0 8px 30px
#17263830;opacity:0;transform:translateY(12px);transition:.2s;pointer-events:none;z-
index:25}.toast.show{opacity:1;transform:translateY(0)}
0008  @media(max-width:1050px){.layout{grid-template-columns:220px 1fr}.content{padding:25px
22px}.topbar{padding:0 22px}.stats{grid-template-columns:repeat(2,minmax(0,1fr))}}
0009  body.auth-required .layout{display:none}.auth-screen{display:none;min-height:100vh;place-
items:center;padding:24px;background:radial-gradient(ellipse at 18% 10%,#1a6052 0,transparent
38%),linear-gradient(135deg,#0b1421,#13283b 58%,#0e3c38);color:#e7eef3}.auth-required .auth-
screen{display:grid}.auth-box{width:min(430px,100%);padding:30px;background:#fff;color:var(--
ink);border:1px solid #dbe5e8;border-radius:10px;box-shadow:0 28px 90px #06111c70;animation:auth-in
.35s ease-out}.auth-brand{display:flex;align-items:center;gap:10px;color:#15283b;font-
weight:800;letter-spacing:.12em}.auth-brand-mark{color:#159d83;font-size:25px}.auth-kicker{margin-
top:29px;color:#159b83;font-size:9px;font-weight:800;letter-spacing:.16em}.auth-box h1{margin:5px 0
5px;font-size:24px}.auth-copy{margin:0 0 22px;color:#7f8b98;font-size:12px}.auth-
form{display:grid;gap:13px}.auth-form label{display:grid;gap:5px;color:#61717e;font-size:10px;font-
weight:700}.auth-form input{width:100%;padding:10px 11px;border:1px solid #dce5e8;border-
radius:6px;outline:none}.auth-form input:focus{border-color:#21aa90;box-shadow:0 0 0 3px
#e6f6f2}.auth-submit{width:100%;margin-top:4px}.auth-switch{margin:18px 0 0;text-
align:center;color:#788793;font-size:11px}.auth-switch
button{border:0;background:none;color:#128b75;font-weight:700}.auth-
error{display:none;margin:0;padding:9px 10px;border-
radius:6px;background:#fbeae8;color:#a84f49;font-size:11px}.auth-error.show{display:block}.auth-
note{margin-top:20px;padding-top:13px;border-top:1px solid #edf1f2;color:#98a3ab;font-size:9px;text-
align:center}@keyframes auth-
in{from{opacity:0;transform:translateY(8px)}to{opacity:1;transform:translateY(0)}}@media(max-
width:500px){.auth-box{padding:23px}}
0010  @media(max-width:720px){.layout{grid-template-columns:66px 1fr}.sidebar{padding:18px
7px}.brand{padding:0 12px 25px}.brand>span:last-child,.workspace-label,.workspace,.caption,.nav-
label,.brief,.profile>span,.profile>b{display:none}.nav button{justify-content:center;padding:10px
0;font-size:0}.nav .ico{font-size:17px}.nav .count{display:none}.side-bottom{margin-
top:auto}.profile{justify-content:center;padding-left:0;padding-
right:0}.topbar{height:60px;padding:0 13px}.crumb{font-size:10px}.top-actions{gap:8px}.live{font-
size:0}.live i{font-size:0}.top-actions .btn{font-size:0;padding:7px}.content{padding:19px
13px}.title-row{align-items:flex-start;flex-direction:column}.title-row h1{font-
size:22px}.actions{width:100%}.overview-grid,.lower-grid{grid-template-
columns:1fr}.stats{gap:8px}.stat{padding:12px;min-height:98px}.stat-value{font-
size:22px}.panel{padding:14px}.chart-wrap{height:180px}}
0011  </style>
0012  <style>.auth-form [hidden]{display:none}</style>
0013  </head>
0014  <body class="auth-required">
0015  <section class="auth-screen" id="authScreen" aria-label="Sign in">
0016    <div class="auth-box">
0017      <div class="auth-brand"><span class="auth-brand-
mark">✳</span><span>NORTHSTAR</span></div>
0018      <div class="auth-kicker">FLIGHT OPERATIONS</div>
0019      <h1 id="authTitle">Welcome back</h1>
0020      <p class="auth-copy" id="authCopy">Sign in to access the airline operations workspace.</p>
0021      <form class="auth-form" id="authForm">
0022        <label id="authNameField" hidden>Name<input name="name" autocomplete="name"
maxlength="80"></label>
0023        <label>Email<input name="email" type="email" autocomplete="email" required
maxlength="254"></label>
```

```
0024        <label>Password<input name="password" type="password" autocomplete="current-password"
required minlength="10"></label>
0025        <p class="auth-error" id="authError" role="alert"></p>
0026        <button class="btn primary auth-submit" id="authSubmit" type="submit">Sign in</button>
0027      </form>
0028      <p class="auth-switch"><span id="authSwitchText">New to Northstar?</span> <button
id="authModeButton" type="button">Create an account</button></p>
0029      <div class="auth-note">Local airline operations workspace</div>
0030    </div>
0031  </section>
0032  <div class="layout">
0033    <aside class="sidebar">
0034      <div class="brand"><span class="brand-mark">✳</span><span>NORTHSTAR<small>FLIGHT
OPERATIONS</small></span></div>
0035      <div class="workspace-label">WORKSPACE</div><div class="workspace"><span class="workspace-
badge">NS</span><span>Northstar Airways<small>Airline logistics demo</small></span></div>
0036      <div class="caption">OPERATIONS</div><nav class="nav" id="nav">
0037        <button class="active" data-page="overview"><span class="ico">◫</span><span class="nav-
label">Overview</span></button>
0038        <button data-page="flights"><span class="ico">↗</span><span class="nav-label">Flight
schedule</span><span class="count" id="flightCount">12</span></button>
0039        <button data-page="fleet"><span class="ico">✈</span><span class="nav-
label">Fleet</span></button>
0040        <button data-page="crew"><span class="ico">♙</span><span class="nav-label">Crew
roster</span></button>
0041        <button data-page="cargo"><span class="ico">▦</span><span class="nav-label">Cargo
tracking</span></button>
0042        <button data-page="passengers"><span class="ico">♔</span><span class="nav-
label">Passengers & loyalty</span></button>
0043      </nav>
0044      <div class="caption">NETWORK WATCH</div><nav class="nav"><button data-
page="disruptions"><span class="ico">⌁</span><span class="nav-label">Disruptions</span><span
class="count alert" id="disruptionCount">2</span></button></nav>
0045      <div class="caption">SUSTAINABILITY</div><nav class="nav"><button data-
page="emissions"><span class="ico">◉</span><span class="nav-label">Emissions</span></button></nav>
0046      <div class="side-bottom"><div class="brief"><span
class="spark">✦</span><strong>Operations brief</strong><p>Review the network snapshot and sample
operational alerts.</p></div><div class="profile"><div class="avatar"
id="profileInitials">NS</div><span id="profileName">Account<small>Signed in</small></span><button
class="btn" id="signOutBtn" type="button" title="Sign out">↪</button></div></div>
0047    </aside>
0048    <main class="main">
0049      <header class="topbar"><div class="crumb">Northstar Airways <span> / </span><b
id="crumb">Overview</b></div><div class="top-actions"><span class="live"
id="connectionStatus"><i></i> CONNECTING</span><button class="btn" id="resetBtn" title="Refresh from
Oracle">↻ Refresh</button><button class="btn" id="notifyBtn" aria-
label="Notifications">♤</button><div class="top-avatar" id="topAvatar">NS</div></div></header>
0050      <section class="content" id="content"></section>
0051    </main>
0052  </div>
0053  <div class="modal-bg" id="modalBg"><section class="modal"><button class="modal-close"
id="modalClose">×</button><div id="modalBody"></div></section></div><div class="toast"
id="toast"></div>
0054  <script>
0055  const seed={
0056   flights:[
0057    {id:'NS 204',route:'DEL → DXB',departure:'09:40',arrival:'12:15',aircraft:'VT-NXA ·
A321neo',gate:'A12',status:'Boarding'},
0058    {id:'NS 118',route:'BOM → SIN',departure:'10:25',arrival:'18:05',aircraft:'VT-NXB · B787-
9',gate:'B06',status:'On time'},
0059    {id:'NS 356',route:'BLR → LHR',departure:'11:10',arrival:'17:45',aircraft:'VT-NXC · A350-
900',gate:'C03',status:'On time'},
0060    {id:'NS 087',route:'DEL → LHR',departure:'12:30',arrival:'18:00',aircraft:'VT-NXD · B787-
9',gate:'A08',status:'Delayed'},
0061    {id:'NS 412',route:'HYD → FRA',departure:'13:05',arrival:'19:10',aircraft:'VT-NXE ·
A321neo',gate:'D02',status:'On time'},
0062    {id:'NS 229',route:'MAA → CMB',departure:'14:20',arrival:'15:45',aircraft:'VT-NXF ·
A320',gate:'B14',status:'Boarding'},
0063    {id:'NS 630',route:'DEL → BOM',departure:'15:00',arrival:'17:10',aircraft:'VT-NXG ·
A320neo',gate:'A04',status:'On time'},
0064    {id:'NS 504',route:'CCU → SIN',departure:'16:40',arrival:'23:00',aircraft:'VT-NXH ·
A321neo',gate:'C08',status:'Scheduled'},
0065    {id:'NS 777',route:'BLR → DXB',departure:'18:15',arrival:'21:05',aircraft:'VT-NXI ·
A321neo',gate:'B09',status:'Scheduled'},
0066    {id:'NS 291',route:'BOM → DEL',departure:'19:30',arrival:'21:40',aircraft:'VT-NXJ ·
A320',gate:'A16',status:'Scheduled'},
0067    {id:'NS 642',route:'DEL → SIN',departure:'21:00',arrival:'05:25',aircraft:'VT-NXB · B787-
9',gate:'A10',status:'Scheduled'},
0068    {id:'NS 094',route:'HYD → BOM',departure:'22:15',arrival:'23:50',aircraft:'VT-NXK ·
A320neo',gate:'D07',status:'Scheduled'}
0069   ],
0070   fleet:[
0071    {id:'VT-NXA',model:'Airbus A321neo',capacity:'24,000 kg',efficiency:'2.8 L / 100
km',status:'Active',flights:'NS 204'},
```

```
0072    {id:'VT-NXB',model:'Boeing 787-9',capacity:'44,000 kg',efficiency:'2.3 L / 100
km',status:'Active',flights:'NS 118 · NS 642'},
0073    {id:'VT-NXC',model:'Airbus A350-900',capacity:'45,000 kg',efficiency:'2.5 L / 100
km',status:'Active',flights:'NS 356'},
0074    {id:'VT-NXD',model:'Boeing 787-9',capacity:'44,000 kg',efficiency:'2.3 L / 100
km',status:'Active',flights:'NS 087'},
0075    {id:'VT-NXE',model:'Airbus A321neo',capacity:'24,000 kg',efficiency:'2.8 L / 100
km',status:'Active',flights:'NS 412'},
0076    {id:'VT-NXF',model:'Airbus A320',capacity:'19,000 kg',efficiency:'3.1 L / 100
km',status:'Maintenance',flights:'—'},
0077    {id:'VT-NXG',model:'Airbus A320neo',capacity:'20,000 kg',efficiency:'2.9 L / 100
km',status:'Active',flights:'NS 630'},
0078    {id:'VT-NXH',model:'Airbus A321neo',capacity:'24,000 kg',efficiency:'2.8 L / 100
km',status:'Active',flights:'NS 504'}
0079   ],
0080   crew:[
0081    {id:'CR-021',name:'Aarav Mehta',role:'Captain',base:'Delhi',flight:'NS 204',shift:'6.2
h',status:'On duty'},
0082    {id:'CR-034',name:'Mira Shah',role:'First Officer',base:'Mumbai',flight:'NS 118',shift:'5.8
h',status:'On duty'},
0083    {id:'CR-048',name:'Kabir Rao',role:'Cabin lead',base:'Bengaluru',flight:'NS 356',shift:'7.1
h',status:'On duty'},
0084    {id:'CR-055',name:'Anaya Iyer',role:'Captain',base:'Delhi',flight:'NS 087',shift:'4.4
h',status:'On duty'},
0085    {id:'CR-069',name:'Ishaan Das',role:'First Officer',base:'Hyderabad',flight:'NS
412',shift:'5.1 h',status:'On duty'},
0086    {id:'CR-073',name:'Sana Kapoor',role:'Cabin crew',base:'Chennai',flight:'NS 229',shift:'3.5
h',status:'Standby'}
0087   ],
0088   cargo:[
0089    {id:'CG-7401',name:'Temperature-controlled biologics',category:'Healthcare',flight:'NS
204',weight:'820 kg',required:'2–8 °C',current:'4.2 °C',tracking:'On track'},
0090    {id:'CG-7402',name:'Fresh seafood consignment',category:'Perishable',flight:'NS
118',weight:'1,260 kg',required:'0–4 °C',current:'3.1 °C',tracking:'On track'},
0091    {id:'CG-7403',name:'Precision instrument cases',category:'High value',flight:'NS
356',weight:'445 kg',required:'Ambient',current:'22 °C',tracking:'On track'},
0092    {id:'CG-7404',name:'Clinical trial samples',category:'Healthcare',flight:'NS
087',weight:'310 kg',required:'2–8 °C',current:'9.1 °C',tracking:'Alert'},
0093    {id:'CG-7405',name:'Specialty produce',category:'Perishable',flight:'NS 412',weight:'970
kg',required:'4–8 °C',current:'6.3 °C',tracking:'On track'}
0094   ],
0095   passengers:[
0096    {id:'P-28104',name:'Aditi Sharma',tier:'Summit',allowance:'+15 kg',baggage:'28
kg',flight:'NS 204'},
0097    {id:'P-28105',name:'Rohan Nair',tier:'Aurora',allowance:'+5 kg',baggage:'22 kg',flight:'NS
118'},
0098    {id:'P-28106',name:'Meera Kulkarni',tier:'Horizon',allowance:'+0 kg',baggage:'18
kg',flight:'NS 356'},
0099    {id:'P-28107',name:'Arjun Patel',tier:'Summit',allowance:'+15 kg',baggage:'31 kg',flight:'NS
087'},
0100    {id:'P-28108',name:'Diya Bose',tier:'Aurora',allowance:'+5 kg',baggage:'20 kg',flight:'NS
412'},
0101    {id:'P-28109',name:'Neel Menon',tier:'Horizon',allowance:'+0 kg',baggage:'17 kg',flight:'NS
229'}
0102   ],
0103   disruptions:[
0104    {id:'DP-5101',flight:'NS 087',route:'DEL → LHR',reason:'Weather restriction',delay:'42
min',date:'10 Oct · 12:30',status:'Monitoring'},
0105    {id:'DP-5102',flight:'NS 412',route:'HYD → FRA',reason:'Ground handling',delay:'18
min',date:'10 Oct · 13:05',status:'Response active'},
0106    {id:'DP-5103',flight:'NS 356',route:'BLR → LHR',reason:'Air traffic flow',delay:'12
min',date:'10 Oct · 11:10',status:'Resolved'}
0107   ],
0108   emissions:[
0109    {id:'EM-8801',flight:'NS 204',route:'DEL → DXB',fuel:'6,840 L',co2:'18.2 t',aircraft:'VT-
NXA',logged:'10 Oct · 12:25'},
0110    {id:'EM-8802',flight:'NS 118',route:'BOM → SIN',fuel:'14,220 L',co2:'37.8 t',aircraft:'VT-
NXB',logged:'10 Oct · 18:10'},
0111    {id:'EM-8803',flight:'NS 356',route:'BLR → LHR',fuel:'22,680 L',co2:'60.3 t',aircraft:'VT-
NXC',logged:'10 Oct · 17:52'},
0112    {id:'EM-8804',flight:'NS 087',route:'DEL → LHR',fuel:'19,240 L',co2:'51.1 t',aircraft:'VT-
NXD',logged:'10 Oct · 18:12'},
0113    {id:'EM-8805',flight:'NS 412',route:'HYD → FRA',fuel:'16,400 L',co2:'43.5 t',aircraft:'VT-
NXE',logged:'10 Oct · 19:20'},
0114    {id:'EM-8806',flight:'NS 229',route:'MAA → CMB',fuel:'4,620 L',co2:'12.3 t',aircraft:'VT-
NXF',logged:'10 Oct · 15:55'}
0115   ]
0116  };
0117  let
data={flights:[],fleet:[],crew:[],cargo:[],passengers:[],disruptions:[],emissions:[],loyalty_tiers:[
]},page='overview',toastTimer,firstRender=true,deferredRender=false;const configs={
0118   flights:{title:'Flight schedule',sub:'Coordinate departures, aircraft and crew across the
network.',icon:'↗',cols:[['Flight','id'],['Route','route'],['Departure','departure'],['Arrival','arr
ival'],['Aircraft','aircraft'],['Status','status']]},
0119   fleet:{title:'Fleet',sub:'Aircraft capability and availability across the
network.',icon:'✈',cols:[['Aircraft ID','id'],['Aircraft
```

```
model','model'],['Capacity','capacity'],['Fuel efficiency','efficiency'],['Today's
flight','flights'],['Status','status']]},
0120   crew:{title:'Crew roster',sub:'Assignments, duty hours and operating
bases.',icon:'♗',cols:[['Crew member','name'],['Role','role'],['Base','base'],['Flight
assignment','flight'],['Shift hours','shift'],['Status','status']]},
0121   cargo:{title:'Cargo tracking',sub:'Special cargo manifests and temperature
checkpoints.',icon:'▦',cols:[['Shipment','name'],['Category','category'],['Flight','flight'],['Weig
ht','weight'],['Required temp.','required'],['Current temp.','current'],['Tracking','tracking']]},
0122   passengers:{title:'Passengers & loyalty',sub:'Passenger baggage details and loyalty tier
allowances.',icon:'♔',cols:[['Passenger','name'],['Passenger ID','id'],['Loyalty
tier','tier'],['Allowance','allowance'],['Baggage','baggage'],['Flight','flight']]},
0123   disruptions:{title:'Disruptions',sub:'Delay events, reason codes and response status linked
to
flights.',icon:'↝',cols:[['Event','id'],['Flight','flight'],['Route','route'],['Reason','reason'],['
Delay','delay'],['Event time','date'],['Status','status']]},
0124   emissions:{title:'Emissions',sub:'Flight-level fuel and CO₂ logs for sustainability
reporting.',icon:'◉',cols:[['Log','id'],['Flight','flight'],['Route','route'],['Fuel
logged','fuel'],['CO₂','co2'],['Aircraft','aircraft'],['Logged at','logged']]}
0125  };
0126  const
content=document.getElementById('content'),crumb=document.getElementById('crumb');function
clone(v){return JSON.parse(JSON.stringify(v))}function esc(v){return
String(v??'').replace(/[&<>"']/g,c=>({'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;',"'":'&#39;'}[c]
))}function toast(msg){let
t=document.getElementById('toast');t.textContent=msg;t.classList.add('show');clearTimeout(toastTimer
);toastTimer=setTimeout(()=>t.classList.remove('show'),2400)}function badge(v){let
s=String(v||'').toLowerCase(),c=s.includes('alert')||s.includes('delay')||s.includes('monitor')?'red
':s.includes('board')||s.includes('response')||s.includes('maint')?'amber':s.includes('active')||s.i
ncludes('track')||s.includes('resolved')||s.includes('duty')?'green':s.includes('scheduled')?'blue':
'gray';return `<span class="badge ${c}">${esc(v)}</span>`}function
table(rows,cols){if(!rows.length)return '<div class="empty">No records match your
search.</div>';return `<div class="table-
wrap"><table><thead><tr>${cols.map(c=>`<th>${esc(c[0])}</th>`).join('')}</tr></thead><tbody>${rows.m
ap((r,i)=>`<tr data-
row="${i}">${cols.map((c,j)=>`<td>${c[1]==='status'||c[1]==='tracking'?badge(r[c[1]]):esc(r[c[1]])}<
/td>`).join('')}</tr>`).join('')}</tbody></table></div>`}
0127  function titleBlock(title,sub,actions=''){return `<div class="title-row"><div><div
class="eyebrow">NORTHSTAR AIRWAYS · ${page==='overview'?'NETWORK CONTROL':'OPERATIONS
CONSOLE'}</div><h1>${esc(title)}</h1><p>${esc(sub)}</p></div><div
class="actions">${actions}</div></div>`}function panelHead(title,sub,link=''){return `<div
class="panel-head"><div><div class="panel-title">${title}</div><div class="panel-
sub">${sub}</div></div>${link}</div>`}
0128  function stats(){let co2=data.emissions.reduce((n,r)=>n+parseFloat(r.co2),0);return `<div
class="stats"><article class="stat"><div class="stat-head">Flights today <span class="stat-
icon">↗</span></div><div class="stat-value">${data.flights.length}</div><div class="stat-
foot">Across ${data.flights.length} flights in Oracle</div></article><article class="stat"><div
class="stat-head">Fleet in service <span class="stat-icon blue">✈</span></div><div class="stat-
value">${data.fleet.filter(x=>String(x.status).toLowerCase()==='active').length} <small style="font-
size:13px;color:#9aa5ad">/ ${data.fleet.length}</small></div><div class="stat-foot">Aircraft
available today</div></article><article class="stat"><div class="stat-head">Open disruptions <span
class="stat-icon amber">↝</span></div><div class="stat-
value">${data.disruptions.filter(x=>!String(x.status).toLowerCase().includes('resolved')).length}</d
iv><div class="stat-foot">Events requiring coordination</div></article><article class="stat"><div
class="stat-head">CO₂ logged <span class="stat-icon red">◉</span></div><div class="stat-
value">${co2.toFixed(1)} <small style="font-size:13px;color:#9aa5ad">t</small></div><div
class="stat-foot">From ${data.emissions.length} Oracle flight logs</div></article></div>`}
0129  function chart(){let vals=[3,5,4,7,6,9,8,11,9],xs=vals.map((_,i)=>54+i*88),ys=vals.map(v=>170-
v*11),path=xs.map((x,i)=>`${i?'L':'M'}${x} ${ys[i]}`).join(' '),area=`${path} L758 180 L54 180
Z`;return `<div class="chart-wrap" id="chartWrap"><svg class="chart" viewBox="0 0 800 220"
preserveAspectRatio="none" aria-label="Flight departures by time"><defs><linearGradient id="area"
x1="0" x2="0" y1="0" y2="1"><stop offset="0" stop-color="#17b394" stop-opacity=".23"/><stop
offset="1" stop-color="#17b394" stop-
opacity="0"/></linearGradient></defs>${[30,70,110,150,190].map(y=>`<line class="gridline" x1="48"
y1="${y}" x2="780"
y2="${y}"/>`).join('')}${['06:00','09:00','12:00','15:00','18:00'].map((x,i)=>`<text x="${54+i*176}"
y="211">${x}</text>`).join('')}<path class="area" d="${area}"/><path class="line"
d="${path}"/>${xs.map((x,i)=>`<circle class="dot" cx="${x}" cy="${ys[i]}"
r="4"/>`).join('')}</svg><div class="tooltip" id="chartTip"></div></div>`}
0130  function overview(){return `${titleBlock('Good morning, team','Here's the latest view across
your airline network.',`<button class="btn" data-export="flights">↓ Export report</button><button
class="btn primary" data-add="flights">+ New flight</button>`)}${stats()}<div class="overview-
grid"><section class="panel">${panelHead('Departure flow','Sample departures · local time','<button
class="text-link" data-page="flights">View schedule →</button>')}${chart()}</section><section
class="panel">${panelHead('Live operations','Network status at a glance')}<div class="status-
list"><div class="status-row"><span class="status-symbol">✈</span><span class="status-
copy"><strong>Flights scheduled</strong><small>Across today's network</small></span><span
class="status-value">${data.flights.length}<small>flights</small></span></div><div class="status-
row"><span class="status-symbol">♗</span><span class="status-copy"><strong>Crew on
duty</strong><small>Assigned across the network</small></span><span class="status-
value">${data.crew.length}<small>members</small></span></div><div class="status-row"><span
class="status-symbol">↝</span><span class="status-copy"><strong>Open
disruptions</strong><small>Events still being monitored</small></span><span class="status-
value">${data.disruptions.filter(x=>!x.status.toLowerCase().includes('resolved')).length}<small>even
ts</small></span></div></div></section></div><section class="panel section-
panel">${panelHead('Upcoming departures','Next scheduled flights','<button class="text-link" data-
```

```
page="flights">All flights
→</button>')}${table(data.flights.slice(0,5),configs.flights.cols)}</section><div class="lower-
grid"><section class="panel">${panelHead('Cold-chain watch','Cargo temperature checkpoints','<button
class="text-link" data-page="cargo">View cargo →</button>')}${data.cargo.slice(0,3).map(c=>`<div
class="mini-row"><span class="status-symbol">▦</span><span
style="flex:1"><strong>${esc(c.name)}</strong><small>${esc(c.id)} · ${esc(c.flight)} ·
${esc(c.current)}</small></span>${badge(c.tracking)}</div>`).join('')}</section><section
class="panel">${panelHead('Disruption desk','Events needing coordination','<button class="text-link"
data-page="disruptions">View all →</button>')}${data.disruptions.slice(0,3).map(d=>`<div
class="mini-row"><span class="mini-mark">!</span><span style="flex:1"><strong>${esc(d.flight)} ·
${esc(d.reason)}</strong><small>${esc(d.route)} · ${esc(d.date)}</small></span><span
class="push">+${esc(d.delay)}</span></div>`).join('')}</section></div>`}
0131  function section(view){let c=configs[view],rows=data[view],statuses=[...new
Set(rows.map(r=>r.status||r.tracking).filter(Boolean))];return `${titleBlock(c.title,c.sub,`<button
class="btn" data-export="${view}">↓ Export CSV</button><button class="btn primary" data-
add="${view}">＋ Add ${view==='flights'?'flight':view==='fleet'?'aircraft':view==='crew'?'crew
member':view==='cargo'?'shipment':view==='passengers'?'passenger':view==='emissions'?'log':'event'}<
/button>`)}${view==='emissions'?`<div class="stats">${emissionStats()}</div>`:''}<section
class="panel section-panel">${panelHead(`<span style="color:#159b83">${c.icon}</span>
 ${c.title}`,`${rows.length} records · Sample data`,`<div class="controls"><input
class="search" id="search" placeholder="⌕  Search ${view}…" type="search"><select
id="filter"><option value="">All
statuses</option>${statuses.map(s=>`<option>${esc(s)}</option>`).join('')}</select></div>`)}<div
id="tableArea">${table(rows,c.cols)}</div></section>${view==='passengers'?`<section
class="panel">${panelHead('Loyalty tier allowances','Sample tier and baggage allowance
reference')}<div class="status-list"><div class="status-row"><span class="status-
symbol">✦</span><span class="status-copy"><strong>Summit</strong><small>Additional baggage
allowance</small></span><span class="status-value">+15 kg</span></div><div class="status-row"><span
class="status-symbol">✦</span><span class="status-copy"><strong>Aurora</strong><small>Additional
baggage allowance</small></span><span class="status-value">+5 kg</span></div><div class="status-
row"><span class="status-symbol">✦</span><span class="status-
copy"><strong>Horizon</strong><small>Additional baggage allowance</small></span><span class="status-
value">+0 kg</span></div></div></section>`:''}`}
0132  function emissionStats(){let fuel=data.emissions.reduce((n,r)=>n+Number(r.fuel.replace(/[
,L]/g,'')),0),co2=data.emissions.reduce((n,r)=>n+Number(r.co2.replace(/[ t]/g,'')),0);return
`<article class="stat"><div class="stat-head">CO₂ logged <span class="stat-icon
red">◉</span></div><div class="stat-value">${co2.toFixed(1)} t</div><div class="stat-foot">Across
sample flight logs</div></article><article class="stat"><div class="stat-head">Emission records
<span class="stat-icon blue">▤</span></div><div class="stat-
value">${data.emissions.length}</div><div class="stat-foot">Illustrative
dataset</div></article><article class="stat"><div class="stat-head">Fuel logged <span class="stat-
icon amber">○</span></div><div class="stat-value">${fuel.toLocaleString()}</div><div class="stat-
foot">Liters across records</div></article><article class="stat"><div class="stat-head">Average per
log <span class="stat-icon">↘</span></div><div class="stat-
value">${(co2/data.emissions.length).toFixed(1)} t</div><div class="stat-foot">Sample CO₂ per
flight</div></article>`}
0133  function render(view='overview'){page=view;document.querySelectorAll('[data-
page]').forEach(b=>b.classList.toggle('active',b.dataset.page===view));crumb.textContent=view===='ove
rview'?'Overview':configs[view].title;content.innerHTML=view==='overview'?overview():section(view);i
f(view==='overview'&&data.flights.length)bindChart();if(view!=='overview')bindFilters(view)}
0134  function bindChart(){let
wrap=document.getElementById('chartWrap'),tip=document.getElementById('chartTip'),svg=wrap.querySele
ctor('svg');svg.addEventListener('pointermove',e=>{let
r=svg.getBoundingClientRect(),ratio=Math.max(0,Math.min(1,(e.clientX-
r.left)/r.width)),idx=Math.min(data.flights.length-1,Math.round(ratio*(data.flights.length-
1))),f=data.flights[idx];if(!f)return;tip.style.left=`${Math.max(12,Math.min(88,ratio*100))}%`;tip.s
tyle.top=`${Math.max(8,e.clientY-r.top-8)}px`;tip.innerHTML=`Flight activity<b>${esc(f.id)} ·
${esc(f.departure)}</b>${esc(f.route)} ·
${esc(f.status)}`;tip.classList.add('show')});svg.addEventListener('pointerleave',()=>tip.classList.
remove('show'))}
0135  function activeRows(view){let
q=(document.getElementById('search')?.value||'').toLowerCase(),status=document.getElementById('filte
r')?.value||'';return
data[view].filter(r=>JSON.stringify(r).toLowerCase().includes(q)&&(!status||(r.status||r.tracking)==
=status))}function
bindFilters(view){document.getElementById('search').addEventListener('input',()=>updateTable(view));
document.getElementById('filter').addEventListener('change',()=>updateTable(view))}function
updateTable(view){document.getElementById('tableArea').innerHTML=table(activeRows(view),configs[view
].cols)}
0136  const formFields={
0137   flights:[['route','Route','text'],['departure','Departure','datetime-
local'],['arrival','Arrival','datetime-
local'],['aircraft_id','Aircraft','fleet'],['status','Status','text']],
0138   fleet:[['model','Aircraft model','text'],['capacity_kg','Capacity
(kg)','number'],['efficiency','Fuel efficiency','number'],['status','Status','text']],
0139   crew:[['name','Crew member','text'],['role','Role','text'],['base','Base location','text']],
0140   cargo:[['flight_id','Flight','flights'],['name','Description','text'],['weight_kg','Weight
(kg)','number'],['category','Category','text'],['required_temp_c','Required temperature
(°C)','number'],['current_temp_c','Current temperature (°C)','number'],['tracking','Tracking
status','text']],
0141   passengers:[['name','Passenger name','text'],['passport_no','Passport
number','text'],['tier_id','Loyalty tier','tiers'],['baggage_kg','Baggage (kg)','number']],
0142   disruptions:[['flight_id','Flight','flights'],['delay_min','Delay
(minutes)','number'],['reason','Reason code','text'],['event_date','Event time','datetime-
local'],['status','Status','text']],
```

```
0143   emissions:[['flight_id','Flight','flights'],['fuel_liters','Fuel
(liters)','number'],['co2_tons','CO₂ (tons)','number'],['logged_at','Logged at','datetime-local']]
0144   };
0145   function inputFor([key,label,type],record={}){let
value=record[key]??(key==='departure'?record.departure_iso:key==='arrival'?record.arrival_iso:'')??'
',required=key!=='db_id'&&key!=='required_temp_c'&&key!=='current_temp_c';if(type==='fleet'||type===
'flights'||type==='tiers'){let opts=type==='fleet'?data.fleet.map(x=>[x.db_id,x.id+' ·
'+x.model]):type==='flights'?data.flights.map(x=>[x.db_id,x.id+' ·
'+x.route]):data.loyalty_tiers.map(x=>[x.tier_id,x.tier_name]);return `<label>${esc(label)}<select
name="${esc(key)}" ${required?'required':''}><option
value="">Choose…</option>${opts.map(([id,text])=>`<option value="${esc(id)}"
${String(value)===String(id)?'selected':''}>${esc(text)}</option>`).join('')}</select></label>`}if(t
ype==='datetime-local'&&value)value=String(value).slice(0,16);return `<label>${esc(label)}<input
name="${esc(key)}" type="${type}" value="${esc(value)}" ${type==='number'?'step="any"':''}
${required?'required':''}></label>`}
0146   async function openRecordForm(view,row=null){let
fields=row?formFields[view]:[['db_id','Numeric ID (leave blank if Oracle generates
it)','number'],...formFields[view]];try{await Promise.all(['flights','fleet','tiers'].map(async
reference=>{data[`ref_${reference}`]=await
apiRequest(`/api/reference/${reference}`)}))}catch(error){toast(error.message);return}document.getEl
ementById('modalBody').innerHTML=`<div class="eyebrow">${esc(configs[view].title.toUpperCase())} ·
ORACLE RECORD</div><h2>${row?'Edit':'Add'} ${esc(configs[view].title.replace(/s$/,''))}</h2><form
id="recordForm"><div class="form-
grid">${fields.map(field=>inputFor(field,row||{})).join('')}</div><div class="modal-actions"><button
type="button" class="btn" id="cancelRecord">Cancel</button><button class="btn primary"
type="submit">${row?'Save changes':'Add
record'}</button></div></form>`;document.getElementById('modalBg').classList.add('open');document.ge
tElementById('cancelRecord').onclick=closeModal;document.getElementById('recordForm').onsubmit=async
e=>{e.preventDefault();let payload={};for(const [key,value] of new
FormData(e.currentTarget).entries()){if(row&&key==='db_id')continue;const
field=fields.find(item=>item[0]===key);payload[key]=value===''?null:field&&(field[2]==='number'||fie
ld[2]==='fleet'||field[2]==='flights'||field[2]==='tiers')?Number(value):value}try{await
apiRequest(`/api/${view}${row?`/${row.db_id}`:''}`,row?'PUT':'POST',payload);closeModal();await
refreshData(true);toast(row?'Oracle record updated.':'Oracle record
added.')}catch(error){toast(error.message)}}}
0147   function showDetail(view,row){let
fields=configs[view].cols;document.getElementById('modalBody').innerHTML=`<div
class="eyebrow">${esc(configs[view].title.toUpperCase())} · ORACLE
RECORD</div><h2>${esc(row.name||row.id||row.flight||'Record
detail')}</h2><p>${esc(row.route||row.model||row.role||row.category||row.reason||'Database
record')}</p><div class="detail-grid">${fields.map(([label,key])=>`<div class="detail-
cell"><small>${esc(label)}</small><strong>${key==='status'||key==='tracking'?badge(row[key]):esc(row
[key])}</strong></div>`).join('')}</div><div class="modal-actions"><button class="btn"
id="deleteRecord">Delete</button><button class="btn" id="editRecord">Edit</button><button class="btn
primary"
id="closeDetail">Done</button></div>`;document.getElementById('modalBg').classList.add('open');docum
ent.getElementById('closeDetail').onclick=closeModal;document.getElementById('editRecord').onclick=(
)=>openRecordForm(view,row);document.getElementById('deleteRecord').onclick=async()=>{if(!confirm(`D
elete ${row.id||row.name||'this record'} from Oracle?`))return;try{await
apiRequest(`/api/${view}/${row.db_id}`,'DELETE');closeModal();await refreshData(true);toast('Oracle
record deleted.')}catch(error){toast(error.message)}}}
0148   function
closeModal(){document.getElementById('modalBg').classList.remove('open');if(deferredRender){deferred
Render=false;render(page)}}
0149   function exportCsv(view){let
cols=configs[view].cols,rows=data[view],csv=[cols.map(x=>x[0]).join(','),...rows.map(r=>cols.map(([,
k])=>`"${String(r[k]??'').replaceAll('"','""')}"`).join(','))].join('\n'),a=document.createElement('
a');a.href=URL.createObjectURL(new Blob([csv],{type:'text/csv'}));a.download=`northstar-
${view}.csv`;a.click();URL.revokeObjectURL(a.href);toast('CSV downloaded.')}
0150   function addRecord(view){openRecordForm(view)}
0151   async function apiRequest(url,method='GET',body){let response;try{response=await
fetch(url,{method,headers:body?{'Content-
Type':'application/json'}:undefined,body:body?JSON.stringify(body):undefined})}catch{throw new
Error('Cannot reach the API. Start the server and open http://127.0.0.1:8001.')}let result=await
response.json().catch(()=>({}));if(!response.ok)throw new Error(result.detail||`Request failed
(${response.status}).`);return result}
0152   async function refreshData(force=false){let next=await
apiRequest('/api/data'),changed=JSON.stringify(next)!==JSON.stringify(data);data=next;document.getEl
ementById('connectionStatus').innerHTML='<i></i> ORACLE
CONNECTED';document.getElementById('connectionStatus').style.color='#258d78';document.getElementById
('flightCount').textContent=data.flights.length;document.getElementById('disruptionCount').textConte
nt=data.disruptions.filter(item=>!String(item.status).toLowerCase().includes('resolved')).length;if(
changed||force){if(document.getElementById('modalBg').classList.contains('open'))deferredRender=true
;else render(page)}return changed}
0153   async function startApp(){try{await
refreshData(true)}catch(error){document.getElementById('connectionStatus').innerHTML='<i></i> DB
OFFLINE';document.getElementById('connectionStatus').style.color='#bd5d57';render('overview');toast(
error.message)}finally{firstRender=false}}
0154   document.addEventListener('click',e=>{let pageButton=e.target.closest('[data-
page]');if(pageButton){render(pageButton.dataset.page);return}let row=e.target.closest('tbody
tr');if(row){let
view=page==='overview'?'flights':page,rows=page==='overview'?data.flights.slice(0,5):activeRows(view
);if(rows[Number(row.dataset.row)])showDetail(view,rows[Number(row.dataset.row)]);return}let
add=e.target.closest('[data-add]');if(add){addRecord(add.dataset.add);return}let
ex=e.target.closest('[data-
export]');if(ex){exportCsv(ex.dataset.export);return}});document.getElementById('modalClose').onclic
k=closeModal;document.getElementById('modalBg').addEventListener('click',e=>{if(e.target.id==='modal
```

```
Bg')closeModal()});document.addEventListener('keydown',e=>{if(e.key==='Escape')closeModal()});docume
nt.getElementById('resetBtn').onclick=()=>{data=clone(seed);render(page);toast('Sample data
reset.')};document.getElementById('notifyBtn').onclick=()=>toast('You're all caught up on
notifications.');document.getElementById('flightCount').textContent=seed.flights.length;document.get
ElementById('disruptionCount').textContent=seed.disruptions.filter(x=>!x.status.toLowerCase().includ
es('resolved')).length;render('overview');
0155  </script>
0156  <script>
0157  let authUser=null,authMode='login';
0158  function setAuthMode(mode){authMode=mode;let
signup=mode==='signup';document.getElementById('authTitle').textContent=signup?'Create your
account':'Welcome back';document.getElementById('authCopy').textContent=signup?'Create a local
account to access flight operations.':'Sign in to access the airline operations
workspace.';document.getElementById('authNameField').hidden=!signup;document.querySelector('#authNam
eField input').required=signup;document.getElementById('authSubmit').textContent=signup?'Create
account':'Sign in';document.getElementById('authSwitchText').textContent=signup?'Already have an
account?':'New to Northstar?';document.getElementById('authModeButton').textContent=signup?'Sign
in':'Create an account';document.querySelector('#authForm
[name="password"]').autocomplete=signup?'new-password':'current-
password';document.getElementById('authError').classList.remove('show')}
0159  function showAuth(message=''){authUser=null;document.body.classList.add('auth-required');let
error=document.getElementById('authError');error.textContent=message;error.classList.toggle('show',B
oolean(message));document.querySelector('#authForm [name="password"]').value=''}
0160  function setSignedInUser(user){authUser=user;document.body.classList.remove('auth-
required');document.getElementById('profileName').innerHTML=`${esc(user.name)}<small>${esc(user.emai
l)}</small>`;let
initials=user.name.trim().split(/\s+/).slice(0,2).map(part=>part[0]).join('').toUpperCase();document
.getElementById('profileInitials').textContent=initials;document.getElementById('topAvatar').textCon
tent=initials}
0161
document.getElementById('authModeButton').onclick=()=>setAuthMode(authMode==='login'?'signup':'login
');
0162  document.getElementById('authForm').onsubmit=async event=>{event.preventDefault();let form=new
FormData(event.currentTarget),payload={email:form.get('email'),password:form.get('password')};if(aut
hMode==='signup')payload.name=form.get('name');let
submit=document.getElementById('authSubmit');submit.disabled=true;try{let user=await
apiRequest(`/api/auth/${authMode}`,'POST',payload);setSignedInUser(user);document.getElementById('au
thForm').reset();await
refreshData(true)}catch(error){showAuth(error.message)}finally{submit.disabled=false}};
0163  document.getElementById('signOutBtn').onclick=async()=>{try{await
apiRequest('/api/auth/logout','POST')}finally{showAuth('You have signed
out.');setAuthMode('login')}};
0164  document.getElementById('resetBtn').onclick=async()=>{try{await
refreshData(true);toast('Refreshed from
Oracle.')}catch(error){document.getElementById('connectionStatus').innerHTML='<i></i> DB
OFFLINE';toast(error.message)}};
0165  async function startApp(){try{let user=await
apiRequest('/api/auth/me');setSignedInUser(user);await
refreshData(true)}catch(error){showAuth(error.message==='Please sign in to continue.'?'':'Oracle
service is unavailable. '+error.message)}}
0166  startApp();
0167  setInterval(()=>{if(authUser)refreshData().catch(error=>{if(error.message.includes('sign
in'))showAuth('Your session expired. Please sign in
again.');else{document.getElementById('connectionStatus').innerHTML='<i></i> DB
OFFLINE';document.getElementById('connectionStatus').style.color='#bd5d57'}})},5000);
0168  </script>
0169  </body>
0170  </html>
```

SCREENSHOTS OF THE APPLICATION WITH DATA

Application and Oracle captures supplied for Review 1. Displayed counts and rows are sample/demo data at the time of capture. The deletion image shows the confirmation dialogue only.

Demo video (Google Drive): shareable URL not supplied. Add the project demonstration link here before submission.

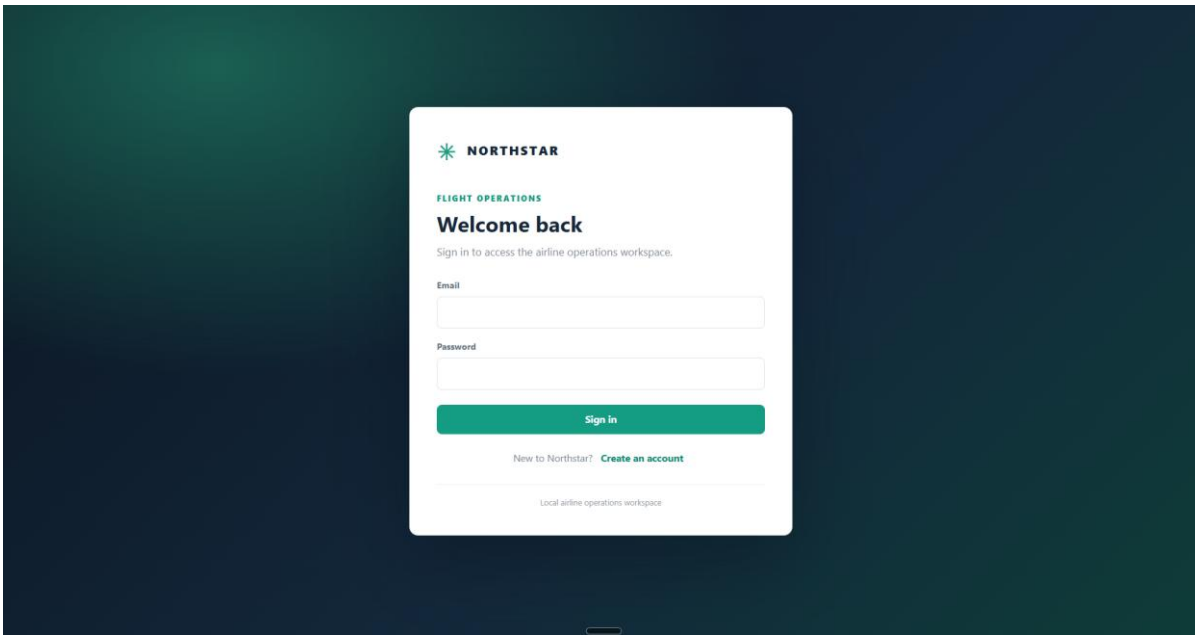


Figure 4. Sign-in page.

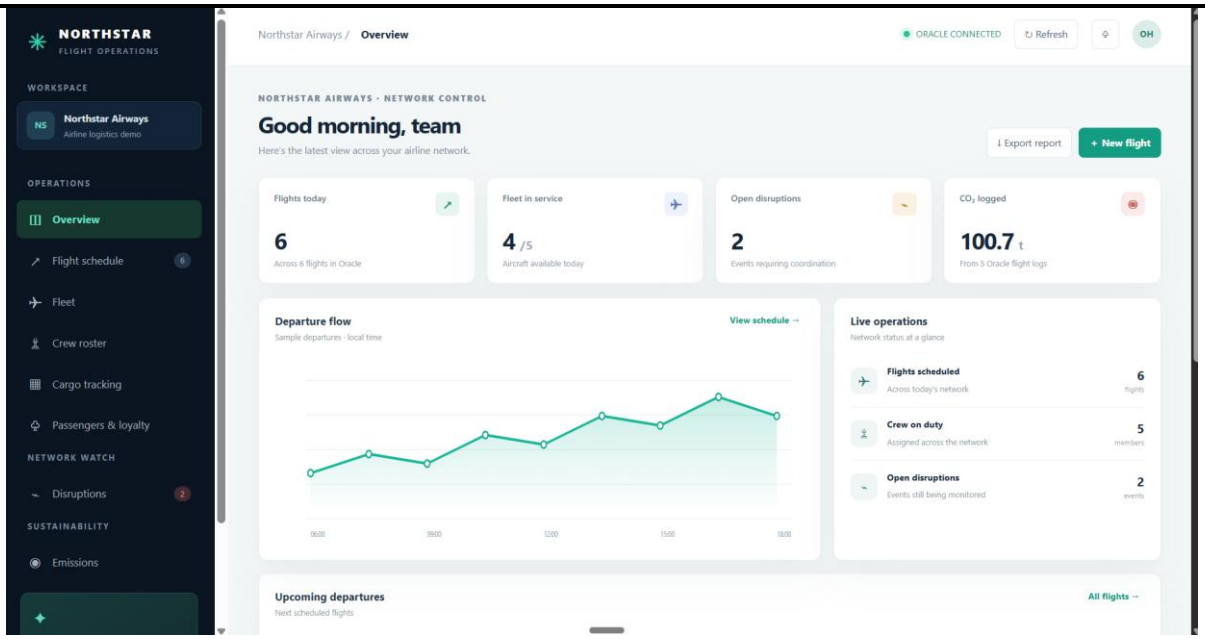


Figure 5. Overview dashboard and sample departure flow.

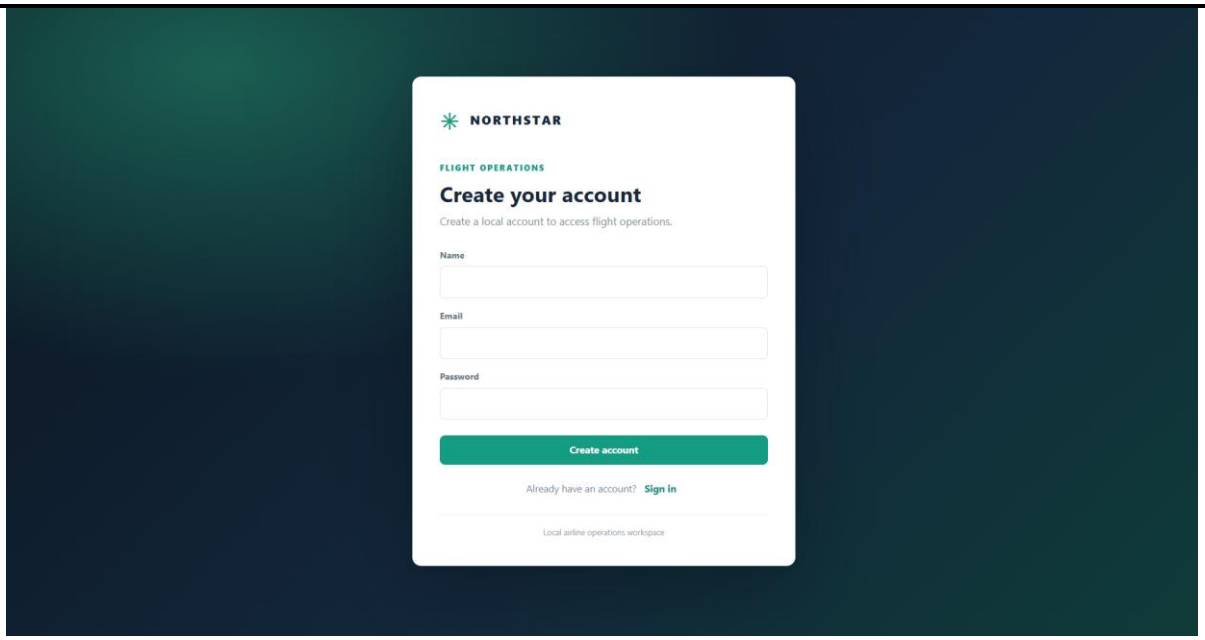


Figure 6. Local account registration page.

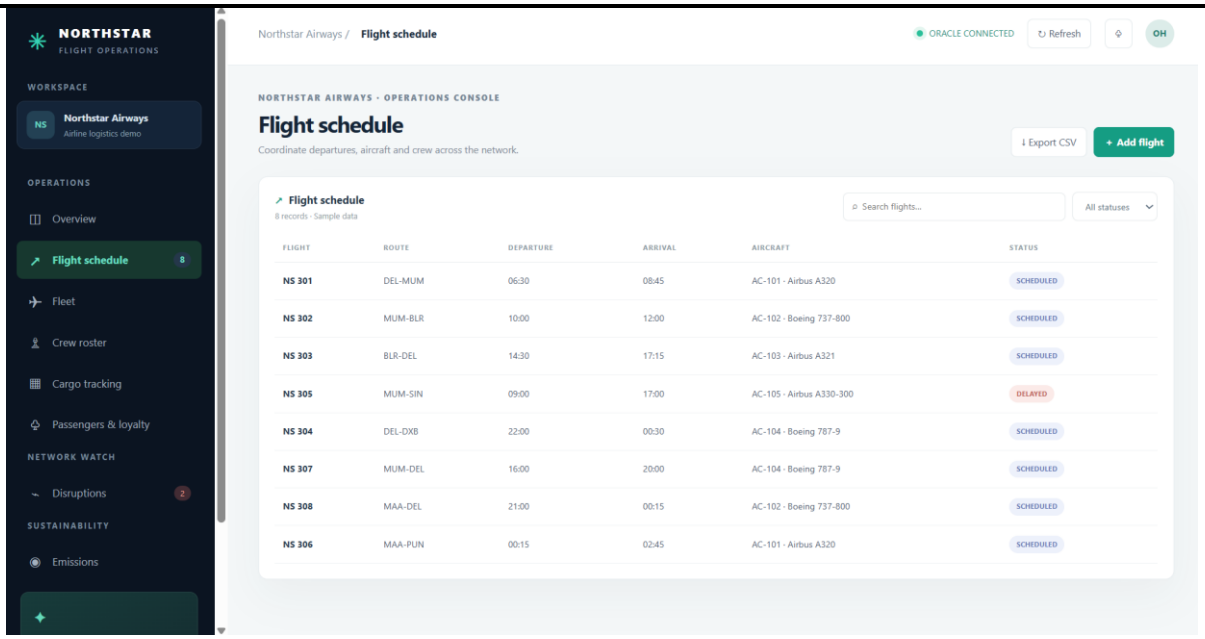


Figure 7. Flight schedule table before insertion.



```
SQL> select * from flight;

 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       381 DEL-MUM
20-AUG-26 06.30.00.000000 AM
20-AUG-26 08.45.00.000000 AM
       101 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       382 MUM-BLR
20-AUG-26 10.00.00.000000 AM
20-AUG-26 12.00.00.000000 PM
       102 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       383 BLR-DEL
20-AUG-26 02.30.00.000000 PM
20-AUG-26 05.15.00.000000 PM
       103 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       384 DEL-DXB
21-AUG-26 10.00.00.000000 PM
22-AUG-26 12.30.00.000000 AM
       104 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       385 MUM-SIN
21-AUG-26 09.00.00.000000 AM
21-AUG-26 05.00.00.000000 PM
       105 DELAYED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       387 MUM-DEL
10-OCT-26 04.00.00.000000 PM
10-OCT-26 08.00.00.000000 PM
       104 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       386 MAA-PUN
30-OCT-26 12.15.00.000000 AM
31-OCT-26 02.45.00.000000 AM
       101 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       388 MAA-DEL
13-OCT-26 09.00.00.000000 PM
14-OCT-26 12.15.00.000000 AM
       102 SCHEDULED

8 rows selected.
```

Figure 8. Oracle flight query before insertion.

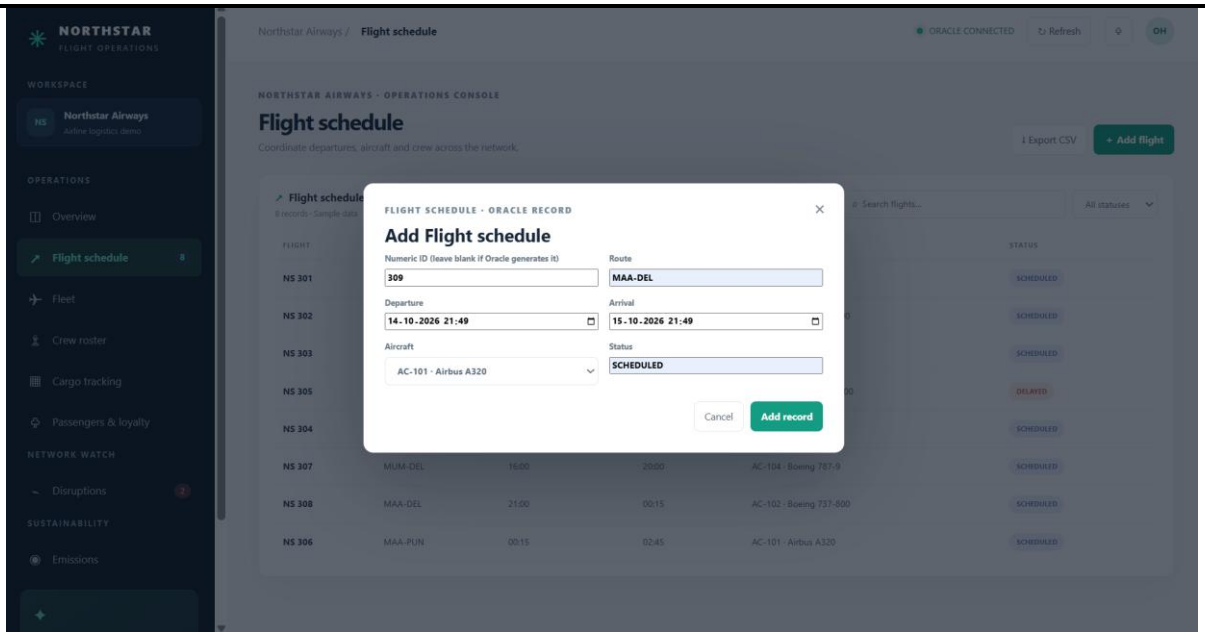


Figure 9. Create flight form for NS 309.



```
SQL> select * from flight;

 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       301 DEL-MUM
20-AUG-26 06.30.00.000000 AM
20-AUG-26 08.45.00.000000 AM
        101 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       302 MUM-BLR
20-AUG-26 10.00.00.000000 AM
20-AUG-26 12.00.00.000000 PM
        102 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       303 BLR-DEL
20-AUG-26 02.30.00.000000 PM
20-AUG-26 05.15.00.000000 PM
        103 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       304 DEL-DXB
21-AUG-26 10.00.00.000000 PM
22-AUG-26 12.30.00.000000 AM
        104 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       305 MUM-SIN
21-AUG-26 09.00.00.000000 AM
21-AUG-26 05.00.00.000000 PM
        105 DELAYED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       307 MUM-DEL
10-OCT-26 04.00.00.000000 PM
10-OCT-26 08.00.00.000000 PM
        104 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       306 MAA-PUN
30-OCT-26 12.15.00.000000 AM
31-OCT-26 02.45.00.000000 AM
        101 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       308 MAA-DEL
13-OCT-26 09.00.00.000000 PM
14-OCT-26 12.15.00.000000 AM
        102 SCHEDULED


 FLIGHT_ID ROUTE_CODE
---------- --------------------
DEP_TIME
---------------------------------------------------------------------------
ARR_TIME
---------------------------------------------------------------------------
AIRCRAFT_ID FLIGHT_STATUS
----------- --------------------
       309 MAA-DEL
14-OCT-26 09.49.00.000000 PM
15-OCT-26 09.49.00.000000 PM
        101 SCHEDULED


9 rows selected.
```

Figure 10. Oracle rows after insertion; nine rows shown.

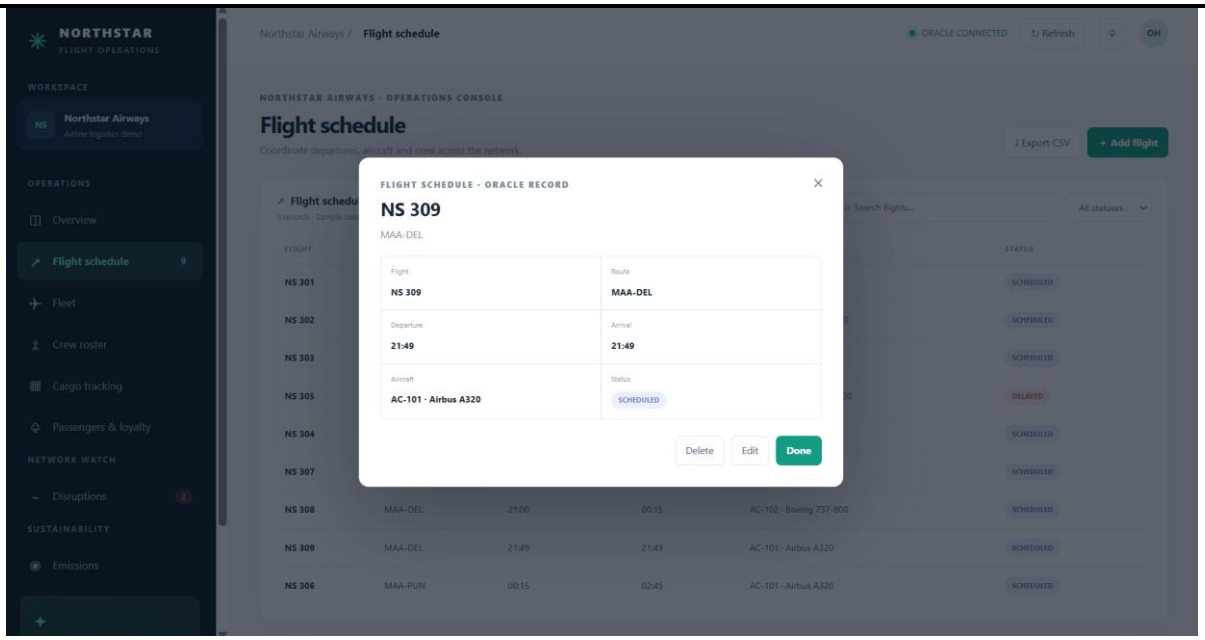


Figure 11. Flight detail view for NS 309.

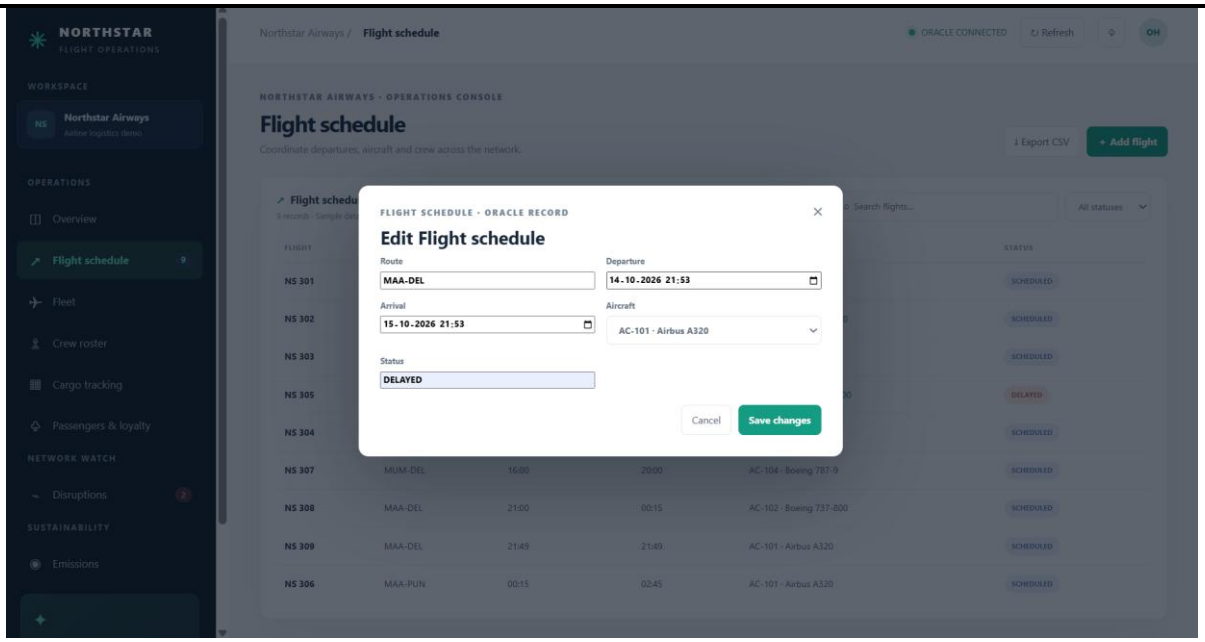


Figure 12. Update form for NS 309, with DELAYED status.

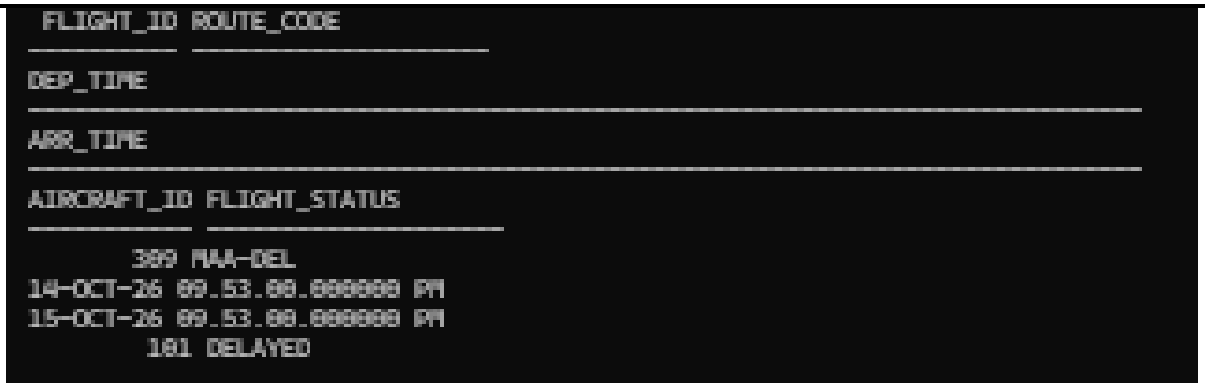


Figure 13. Oracle record after update.

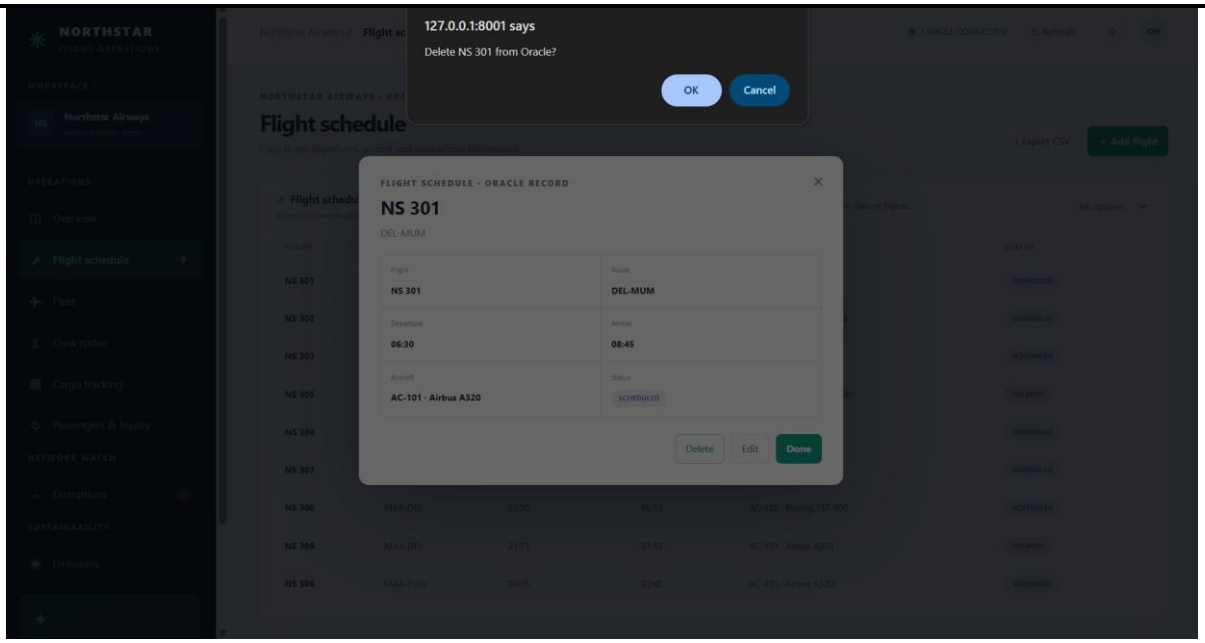


Figure 14. Delete confirmation dialogue for NS 301.

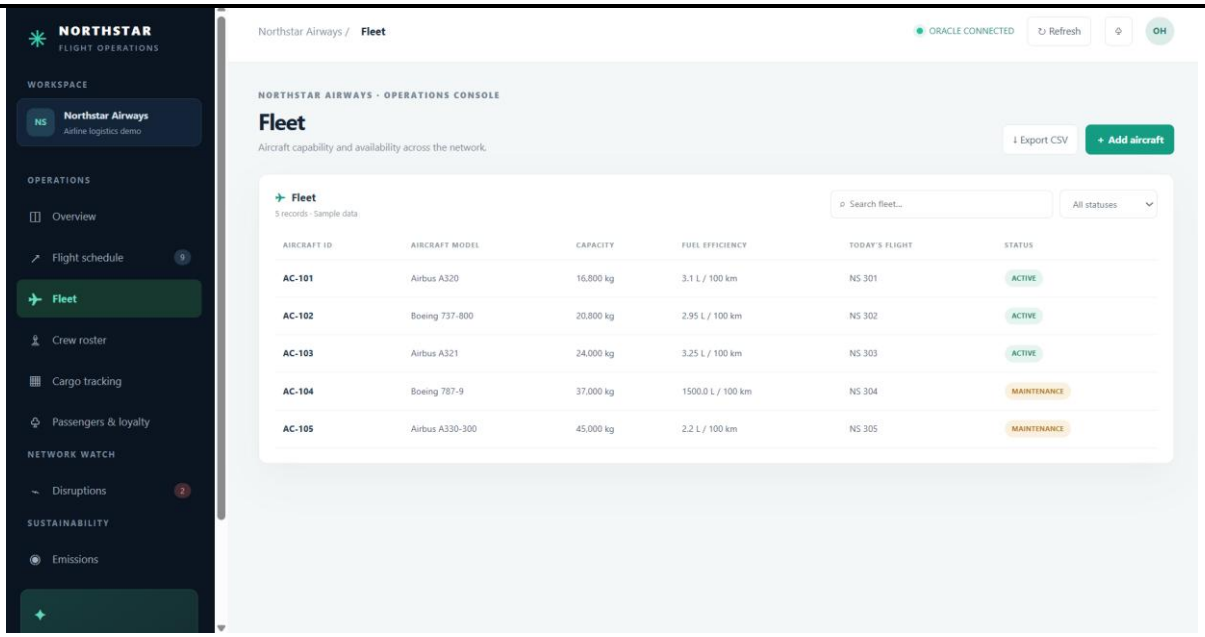


Figure 15. Fleet module.

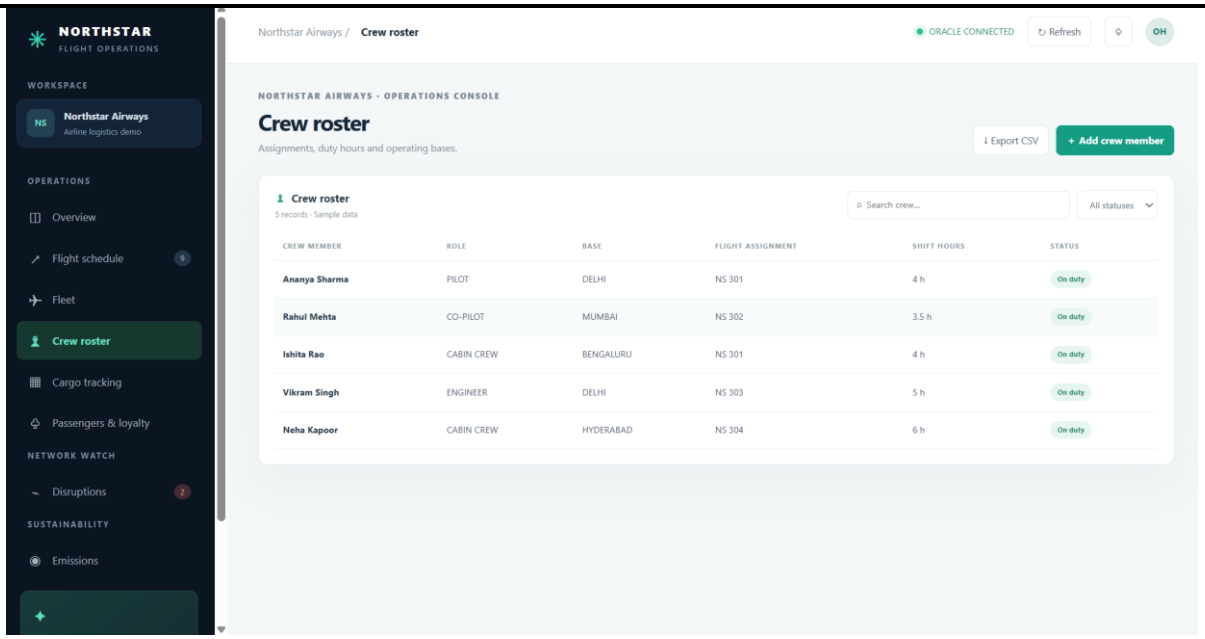


Figure 16. Crew roster module.

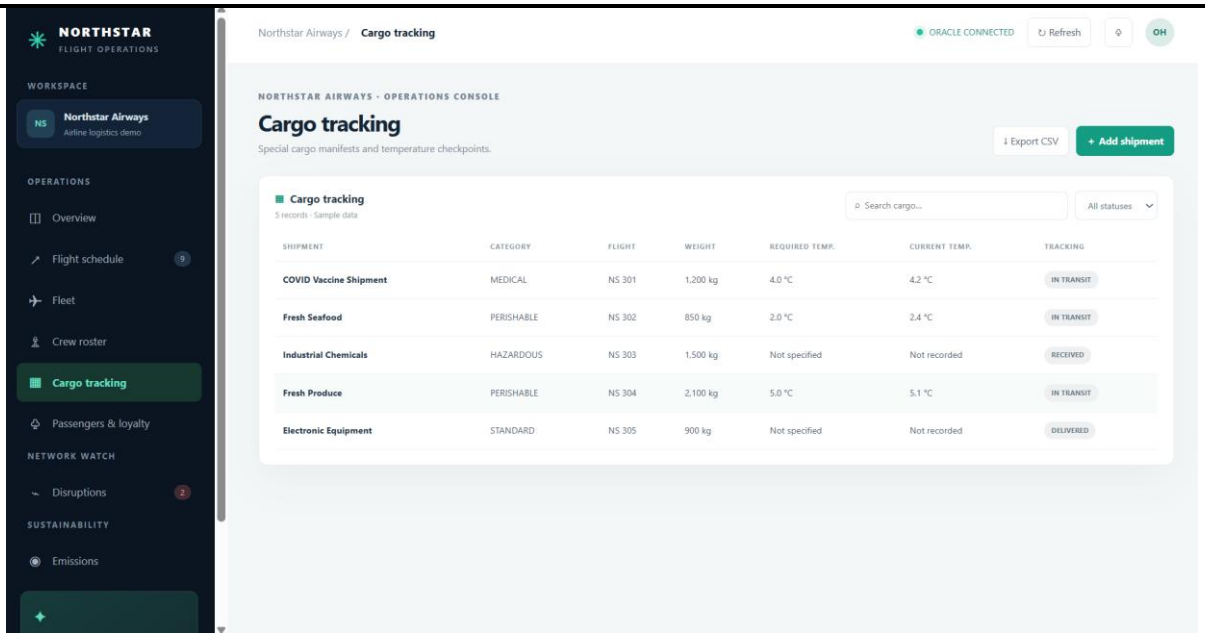


Figure 17. Cargo tracking module.

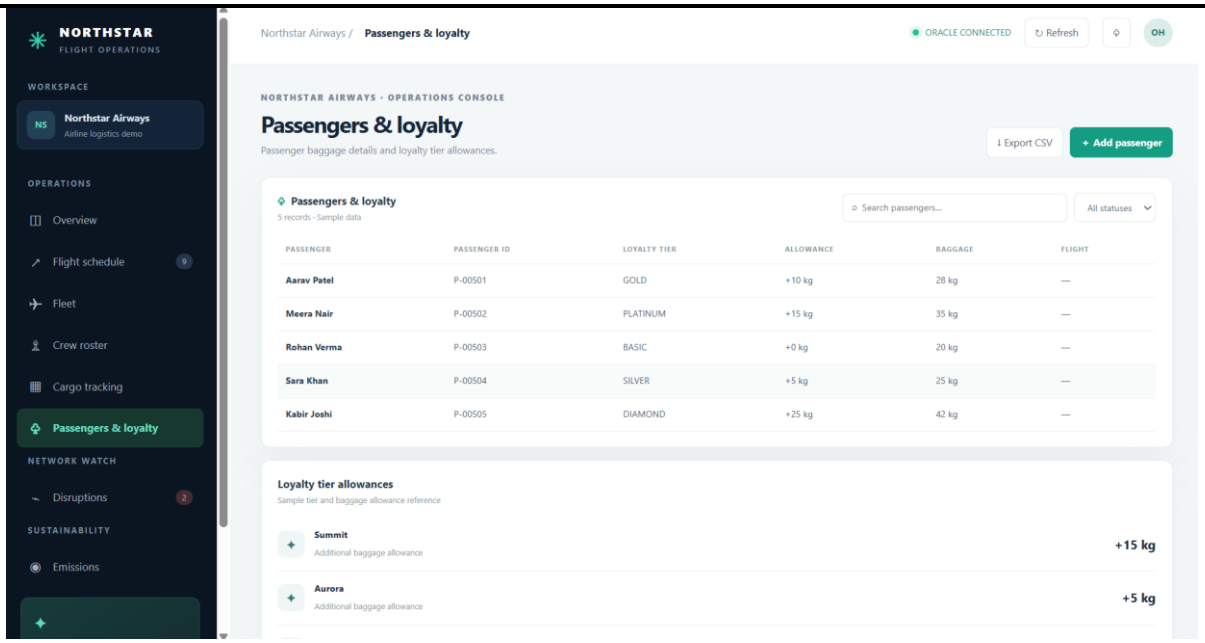


Figure 18. Passengers and loyalty module.

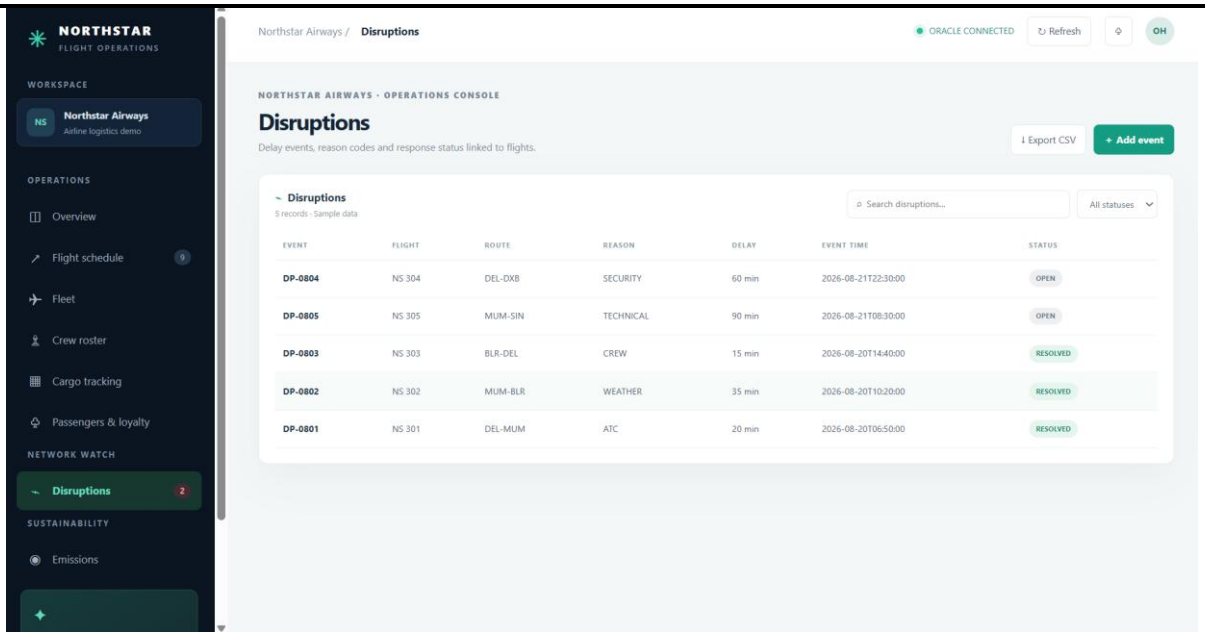


Figure 19. Disruptions module.

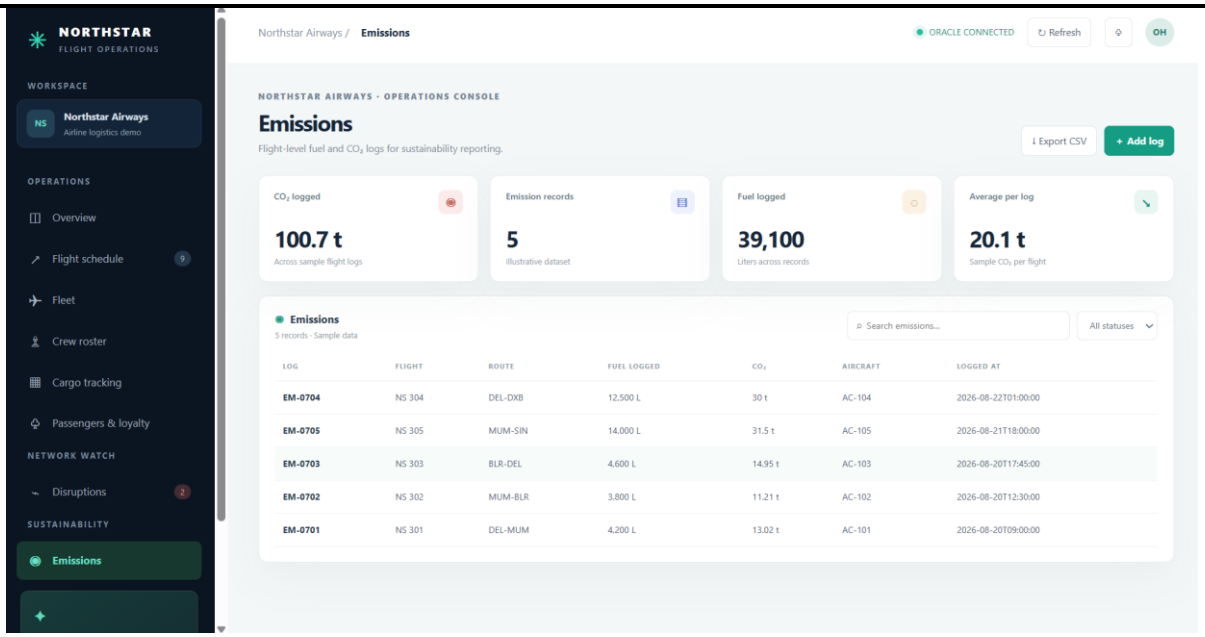


Figure 20. Emissions module.

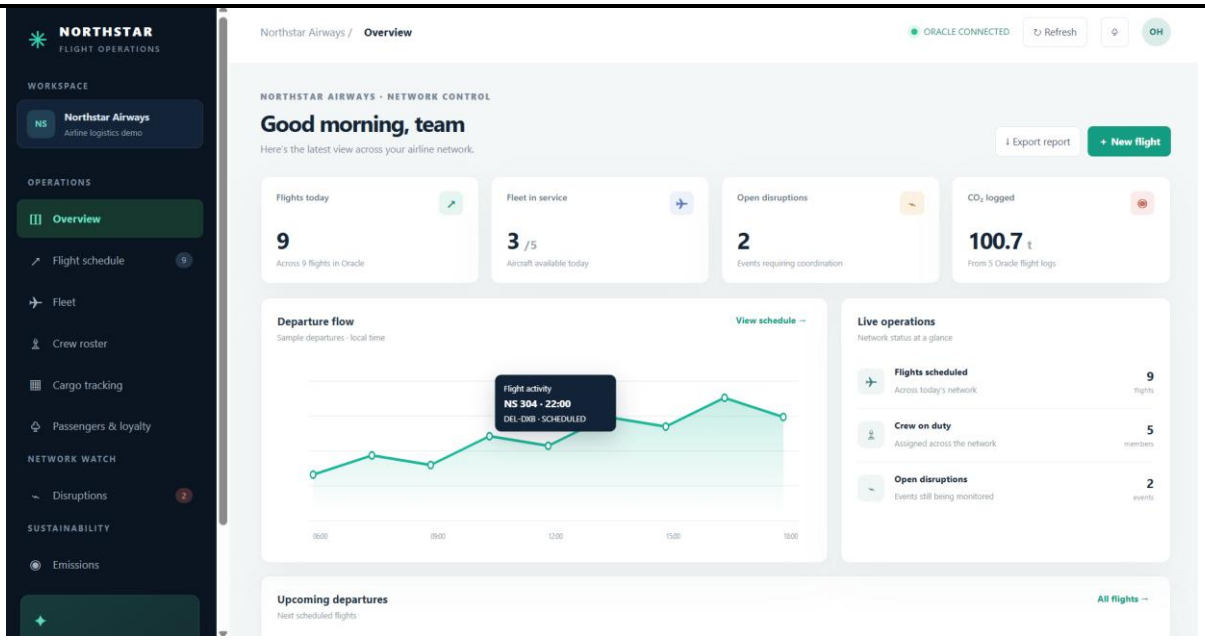


Figure 21. Overview dashboard after the additional flight.

CONCLUSION AND FUTURE SCOPE

The project has progressed from the Review 0 relational design to a local working application connected to Oracle XE. The browser workspace brings key airline records together and demonstrates authentication, flight schedule management, CRUD operations for the demonstrated flight, and module views for fleet, crew, cargo, passengers, disruptions and emissions. The data model uses relational keys and supports sustainability-related records.

The project remains a coursework proof of concept. The supplied dataset is illustrative, the backend is intended for localhost, all registered accounts share the same permissions, and sustainability outcomes have not been quantified. The supplied delete screenshot records the confirmation prompt, not the resulting database state.

Future enhancements

Priority	Enhancement	Expected outcome
1	Role-based access, stronger session controls, TLS, audit logs and CSRF protection	Safer deployment and accountability.
2	Validation, pagination, transaction handling and clearer database errors	More dependable workflows at larger scale.
3	Crew assignment screen with duty/rest validation	Complete staffing workflow.
4	Sensor ingestion and temperature threshold alerts	Faster response to cold-chain excursions.
5	Verified emissions factors and normalized route/load reports	Traceable sustainability analysis.
6	Disruption alerts and historical cause analysis	Improved coordination and operational learning.
7	Automated API/browser checks, backups, migration and monitoring	Safer releases and recovery.
8	Approved airport, booking and maintenance integrations	Fewer duplicate records and broader network view.

A practical next step is to validate every deployed table and key against Oracle metadata, then verify create, update, delete and unauthorized-access cases. Before any shared-network deployment, add roles and audit records. A limited pilot can then measure data completeness, exception-response time and normalized fuel/emissions indicators against an agreed baseline.